Agent-Chameleon: Dynamic Protocol Morphing, Goal-Conditioned Schema Permutation, and Zero-Knowledge Session Handshakes for Multi-Agent Swarm Communications

Aniket Agrawal
Google Cloud AI
Google
aniketagrawal@google.com

***Abstract*—As enterprise artificial intelligence architectures transition from monolithic prompt-response pipelines to distributed multi-agent swarms, autonomous agents increasingly coordinate high-stakes operations across financial enterprise resource planning (ERP) ledgers, clinical healthcare repositories, cloud identity and access management (IAM) planes, and real-time payment settlement rails. Standardized agent communication protocols—most notably the Model Context Protocol (MCP) and Agent2Agent (A2A) operating over JSON-RPC 2.0, HTTP/2, and gRPC—rely on static, human-readable wire schemas and long-lived bearer tokens. We demonstrate that even when payloads are encapsulated within transport-layer encryption or routed through multi-tenant service-mesh proxies that terminate Layer-7 traffic, static agent protocols remain severely vulnerable to statistical traffic fingerprinting, automated schema scraping, and credential replay. Across $N = 1{,}000$ enterprise MCP and A2A tool traces spanning 20 distinct tool classes, standard passive traffic classifiers (Random Forest, Multi-Layer Perceptron, and k-Nearest Neighbors) achieve a 96.5% ensemble mean accuracy (95.0% Random Forest, 94.5% MLP, 100.0% k-NN) in inferring the exact enterprise tool schema invoked by an agent.**

To eliminate these vulnerabilities at line rate, we present Agent-Chameleon, a three-stage polymorphic wire-transport and zero-knowledge session-handshake engine for multi-agent swarm communications. Agent-Chameleon introduces: (1) an Ephemeral X25519/HKDF-SHA256 Session Ratchet and Polymorphic Wire Serialization Engine (`morph_engine.py`) that advances a forward-secure state $R_t = \text{HKDF-SHA256}(R_{t-1}, \textbf{epoch}_t \parallel \textbf{digest}_{t-1}, H(\mathcal{G}_t))$ on every turn and deterministically rotates wire serializations across four structurally distinct codecs—compact binary Protobuf base-128 varints, 4-byte aligned FlatBuffers vtables, RFC 8949 Concise Binary Object Representation (CBOR), and canonical HMAC-signed JSON-RPC—emitting observable `Content-Type: application/x-chameleon-morph+octet-stream` and `X-Chameleon-Epoch-ID` headers; (2) a Goal-Conditioned Bijective Field Permutation and Constant-Rate Chaff Entropy Engine (`feistel_schema.py`) that permutes 16-bit schema field identifiers through a 4-round SHAKE-256 keyed Luby-Rackoff Feistel network $F_k : \{0, \ldots, 2^{16} - 1\} \to \{0, \ldots, 2^{16} - 1\}$ with closed-form inverse F_k^{-1} ($16.58\,\mu$s mean latency, 100.0% lossless bijectivity) and pads XOR-whitened frames into discrete bucket quanta via set-complement and $\mathbb{Z}_{256}$ affine-bijection chaff blocks so single-frame Shannon byte entropy reaches $H(X) = 7.992 \pm 0.005$ bits/byte (≥ 7.980 bits/byte across 100% of frames); and (3) a Sub-35-Microsecond Non-Interactive Fiat-Shamir Schnorr Zero-Knowledge Challenge-Response (`zk_handshake.py`) carried in `X-Chameleon-Proof-Nonce` that proves possession of the ratcheted turn witness without transmitting static API keys or OAuth bearer tokens ($20.92\,\mu$s mean proof generation, $28.64\,\mu$s mean verification, and $28.86\,\mu$s combined hot-path ratchet plus proof latency). Empirical evaluation over $N = 1{,}000$ traces demonstrates that Agent-Chameleon collapses attacker schema-inference accuracy from 96.5% ensemble mean down to 4.67% (5.0% Random Forest, 3.0% MLP, 6.0% k-NN—statistically indistinguishable from the $1/20 = 5.0\%$ uniform random-guess baseline) while achieving 100.0% rejection of token replays, epoch rollbacks, cross-goal injections, and man-in-the-middle bit-flip tampering.

***Index Terms*—Multi-Agent Systems, Protocol Morphing, Feistel Networks, Format-Preserving Encryption, Zero-Knowledge Proofs, Schnorr Identification, Traffic Analysis Resistance, Model Context Protocol, Agent2Agent.**

I. Introduction

The rapid proliferation of autonomous Large Language Model (LLM) agents across enterprise cloud environments has catalyzed a shift from isolated single-model inference toward collaborative multi-agent swarms. In modern production architectures—such as workflows built on the Google Agent Development Kit (ADK), the Agent2Agent (A2A) interoperability specification, and the Model Context Protocol (MCP)—specialized orchestrator, planner, financial auditor, and infrastructure execution agents exchange hundreds of remote procedure calls (RPCs) per workflow to complete complex business transactions [27]–[29]. Each inter-agent message or tool invocation typically traverses heterogeneous trust boundaries, including multi-tenant Kubernetes ingress controllers, Layer-7 (L7) Envoy/Istio service-mesh sidecars, API gateways, and cross-cloud interconnects.

Despite advances in intra-agent prompt-injection defenses [30] and transport-layer encryption [10], the wire protocols underlying multi-agent swarm communications remain structurally static. Specifically, both MCP and A2A standardize on human-readable JSON-RPC 2.0 envelopes containing fixed structural field identifiers (`"jsonrpc"`, `"method"`, `"params"`, `"tool_name"`, `"arguments"`,

"`session_id`") and unpadded variable-length payloads. This static wire design introduces three acute security vulnerabilities in production multi-agent deployments:

1) **Vulnerability to Statistical Traffic Fingerprinting and Tool-Schema Reconnaissance:** Classic work in network traffic analysis demonstrates that variable-length encrypted records and static application schemas leak substantial side-channel information regarding application state and remote procedure identity [15], [16], [18], [19]. Furthermore, in modern enterprise service meshes where L7 observability sidecars, API gateways, or shared message brokers terminate TLS to perform routing and load balancing, raw or lightly framed JSON-RPC payloads are exposed directly to co-resident tenants or compromised middleboxes. Our empirical audit across $N = 1{,}000$ MCP/A2A tool invocations over 20 enterprise tool classes reveals that standard unpadded JSON-RPC exhibits a mean Shannon byte entropy of only 4.788 bits/byte (out of 8.000 bits/byte [20]), enabling lightweight Random Forest, Multi-Layer Perceptron (MLP), and k-Nearest Neighbor (k-NN) classifiers to identify the exact tool schema being invoked with 95.0%, 94.5%, and 100.0% accuracy (96.5% ensemble mean), respectively. Even when JSON-RPC payloads are length-padded to fixed 512-byte multiples, L7 delimiter densities and byte histograms still allow 89.5% Random Forest and 100.0% k-NN classification accuracy.
2) **Automated Schema Scraping and Cross-Goal Payload Forgery:** Because field names and wire encodings remain invariant across turns and sessions, an adversary who observes or scrapes a single valid tool invocation frame immediately learns the structural AST template required to synthesize forged requests. Worse still, static schemas fail to cryptographically bind the wire representation of a tool call to the active high-level reasoning goal $\mathcal{G}_t$ authorized by the upstream orchestrator. Consequently, a compromised sub-agent or man-in-the-middle (MITM) relay can repurpose captured field structures from a benign read-only query goal into a state-mutating financial disbursement goal [28], [29].
3) **Static Bearer Token Leakage vs. Multi-Round Handshake Latency:** Existing multi-agent runtimes predominantly authenticate inter-agent RPCs by attaching static API keys, OAuth 2.0 bearer tokens, or long-lived JWTs in HTTP headers [11]–[13]. If a bearer token is intercepted at an L7 proxy or leaked into debug telemetry, an adversary can replay it across arbitrary tool endpoints until expiration. Conversely, replacing static tokens with per-turn mutual TLS (mTLS) re-handshakes [10] or classical 3-move interactive zero-knowledge identification protocols [2], [24] introduces $450\,\mu$s to $1{,}200\,\mu$s of round-trip and asymmetric PKI overhead per turn, degrading multi-agent swarm throughput by an order of magnitude.

A. Contributions

To resolve the tension between wire-level schema confidentiality, zero-knowledge session authentication, and microsecond-scale multi-agent throughput, we design, implement, and evaluate **Agent-Chameleon**. Our work makes four primary contributions:

1) **Ephemeral X25519/HKDF Session Ratchet and 4-Way Polymorphic Wire Serialization Engine (Stage 1, Section IV):** We formulate a synchronous, forward-secure session ratchet driven by RFC 5869 HKDF-SHA256 [4] and bootstrapped via RFC 7748 X25519 Diffie-Hellman key agreement [6]. On every RPC turn t, the ratchet state R_t evolves as a cryptographic function of the prior key R_{t-1}, the monotonic epoch counter epoch_t, the SHA-256 digest of the preceding wire frame digest_{t-1}, and the active task goal digest $H(\mathcal{G}_t)$. Seeded by R_t, the engine deterministically rotates the wire serialization codec on every turn across four structurally distinct formats with an exact 25% stationary distribution over every 4-epoch window: (i) compact binary Protobuf-style base-128 varints, (ii) 4-byte aligned FlatBuffers vtables (`FBCH`), (iii) RFC 8949 CBOR binary maps [5], and (iv) canonical sorted hex-keyed JSON-RPC with an 8-byte HMAC-SHA256 trailer.
2) **Goal-Conditioned 4-Round Keyed Feistel Schema Permutation and Constant-Rate Chaff Entropy Engine (Stage 2, Section V):** We map canonical MCP/A2A schema field names and dynamic custom symbols into 16-bit integer slots and permute them on every turn via a balanced 4-round Luby-Rackoff Feistel network $F_k : \{0, \ldots, 2^{16} - 1\} \to \{0, \ldots, 2^{16} - 1\}$ [1], [21], [22] keyed by FIPS 202 SHAKE-256 [9] round tables derived from $(R_t, H(\mathcal{G}_t))$. We prove in Theorem 1 that F_k is an exact bijection across all $65{,}536$ domain elements with closed-form inverse F_k^{-1} executing in $16.58\,\mu$s mean latency ($15.83\,\mu$s p50), while exhibiting $> 99.9\%$ avalanche when the goal context $\mathcal{G}_t$ changes. To eliminate packet-length and byte-histogram side channels, we introduce a C-accelerated SHAKE-256 stream whitener paired with Set-Complement and $\mathbb{Z}_{256}$ Affine-Bijection Constant-Rate Chaff Injection into discrete quantum tiers $\{512, 1024, 2048, 3072, 4096, 6144, 8192\}$ bytes. We prove in Theorem 2 that this construction guarantees a single-frame plug-in Shannon byte entropy lower bound $H(X) \geq 7.980$ bits/byte, achieving an empirical mean of 7.992 ± 0.005 bits/byte across $N = 1{,}000$ traces.
3) **Sub-35-Microsecond Non-Interactive Fiat-Shamir Schnorr Zero-Knowledge Handshake (Stage 3, Section VI):** We eliminate static wire bearer tokens by embedding a single-move non-interactive Fiat-Shamir Schnorr Zero-Knowledge Proof of Knowledge (NIZKPoK) [2], [3], [25] in the `X-Chameleon-Proof-Nonce` header. We design a dual-tier verification architecture: a Tier-1 Hot-Path

Ephemeral Ratchet Schnorr NIZKPoK operating over the Mersenne prime field $\mathbb{F}_{M_{127}}$ ($M_{127} = 2^{127} - 1$) with single-turn ephemeral witnesses refreshed every RPC step and bound to a 128-bit Blake2s ratchet commitment tag (20.92 μs mean prove, 28.64 μs mean verify, 30.13 μs p99 prove), paired with Tier-2 full 256-bit prime-field Schnorr ($\mathbb{F}_{2^{255}-19}$) and RFC 8032 Ed25519 [7] epoch anchor signatures for cross-domain auditability. We prove soundness, zero-knowledge simulatability, and replay/desynchronization immunity with automatic ratchet rollback in Theorem 3.

4) **Production-Grade Implementation and Empirical Evaluation (Sections VII–VIII):** We implement Agent-Chameleon as a modular Python package (`agent_chameleon`) verified by a 24-test unit and property suite (100% pass rate) and benchmarked across $N = 1{,}000$ multi-agent traces spanning 20 enterprise tool classes. Agent-Chameleon reduces attacker traffic-classifier accuracy from 96.5% ensemble mean (95.0% Random Forest) to 4.67% (5.0% Random Forest, 3.0% MLP, 6.0% k-NN, matching the 5.0% uniform random-guess baseline), blocks 100.0% of replay and MITM bit-flip attacks, and sustains 1.265×10^6 RPC/s aggregate throughput across 128 concurrent agent sessions.

II. Related Work

A. Agent Communication Protocols and Request Authentication

The Model Context Protocol (MCP) and Agent2Agent (A2A) specifications have established JSON-RPC 2.0 over HTTP/2 and Server-Sent Events (SSE) as the de facto standard for tool invocation and inter-agent task delegation. To authenticate requests across service boundaries, cloud deployments commonly rely on OAuth 2.0 Demonstrating Proof-of-Possession (DPoP, RFC 9449) [11], HTTP Message Signatures (RFC 9421) [12], or contextual HMAC capability tokens such as Macaroons [13]. Concurrently, system-level agent security frameworks such as AgentDojo [28], InjecAgent [29], and CaMeL [30] address indirect prompt injection by isolating control flow from untrusted data streams [27]. However, existing agent protocols and capability tokens leave the wire serialization format and JSON-RPC schema keys completely static across turns. Furthermore, DPoP and RFC 9421 transmit explicit token headers and public-key signatures that cost 180–750 μs per verification in software PKI without concealing the underlying tool schema from L7 traffic analysis.

B. Traffic Analysis Resistance and Format-Transforming Encryption

Statistical traffic analysis has long been shown to undermine encrypted protocols by exploiting packet sizes, timing, and byte distributions [15], [16], [18], [19]. Wright et al. [15] proposed convex optimization for traffic morphing by splitting or padding packets to mimic target length distributions. However, Dyer et al. [16] demonstrated in their "Peek-a-Boo" study that standard length-padding and morphing countermeasures fail against coarse summary classifiers when application framing invariants remain correlated with message types. Later, Dyer et al. [17] introduced Format-Transforming Encryption (FTE) to shape ciphertext streams into regular-expression grammars to bypass deep-packet inspection (DPI) middleboxes. While FTE targets censorship circumvention by making encrypted streams look like static HTTP or SMB regexes, enterprise multi-agent swarms face the inverse problem: preventing an adversary or untrusted L7 proxy from inferring which of $|\mathcal{C}|$ enterprise tool schemas an agent is executing while preserving deterministic, zero-copy-compatible AST deserialization on the authorized receiver.

C. Feistel Format-Preserving Encryption and Cryptographic Ratchets

Constructing pseudorandom permutations (PRPs) over small or arbitrary finite domains was pioneered by Luby and Rackoff [1], extended to arbitrary finite domains by Black and Rogaway [21], formalized as Format-Preserving Encryption (FPE) by Bellare et al. [22], and analyzed at higher rounds by Patarin [23]. Agent-Chameleon adapts the balanced Luby-Rackoff Feistel paradigm specifically to 16-bit AST schema slot identifiers $\{0, \ldots, 2^{16} - 1\}$, conditioning the round PRFs on both an evolving HKDF ratchet state [4] and the active agent goal digest $H(\mathcal{G}_t)$. Unlike the Signal Double Ratchet [14], which encrypts a fixed protobuf envelope for asynchronous human messaging, Agent-Chameleon uses its synchronous HKDF ratchet to simultaneously drive (i) 4-way wire-codec polymorphism, (ii) goal-conditioned Feistel schema tag permutation, (iii) set-complement plus affine-bijection constant-rate chaff synthesis [8], [9], and (iv) single-move Fiat-Shamir Schnorr zero-knowledge proofs [2], [3], [24]–[26].

III. System Model, Threat Model, and Design Goals

A. Multi-Agent Swarm Communication Model

Consider a distributed multi-agent swarm $\mathcal{S} = \{\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_M\}$ executing a directed task workflow across enterprise cloud infrastructure. When an initiator agent $\mathcal{A}_i$ invokes a tool or delegates a sub-task to a responder agent or FastMCP tool server $\mathcal{A}_j$ at discrete turn $t \in \{0, 1, 2, \ldots\}$, the application logic constructs a structured message payload P_t conforming to one of $|\mathcal{C}|$ enterprise tool schemas $\mathcal{C} = \{\mathcal{S}_1, \ldots, \mathcal{S}_{|\mathcal{C}|}\}$ under an active high-level reasoning goal $\mathcal{G}_t \in \{0,1\}^*$.

Formally, each payload P_t is a hierarchical key-value abstract syntax tree (AST) whose internal dictionary nodes map string field identifiers $f \in \mathcal{F}$ (such as `"jsonrpc"`, `"method"`, `"params"`, `"tool_name"`, `"arguments"`, or tool-specific parameter keys) to primitive values ($v \in \mathbb{Z} \cup \mathbb{R} \cup \{\text{true}, \text{false}, \text{null}\} \cup \Sigma^*_{\text{utf8}} \cup \{0,1\}^*$), ordered lists, or nested dictionary subtrees.

Definition 1 (Adversarial Observation and Manipulation Surface)**.** *We consider a probabilistic polynomial-time (PPT) Dolev-Yao and statistical traffic-analysis adversary* $\mathcal{A} = (\mathcal{A}_{clf}, \mathcal{A}_{mitm})$ *positioned along the network or L7 service-mesh path between* $\mathcal{A}_i$ *and* $\mathcal{A}_j$*:*

1) ***Passive Schema-Fingerprinting Adversary ($\mathcal{A}_{clf}$):*** *For each wire packet $W_t = (\mathcal{H}_t, B_t)$ comprising transport metadata headers $\mathcal{H}_t$ and byte payload $B_t \in \{0, \ldots, 255\}^{L_t}$, $\mathcal{A}_{clf}$ extracts a feature vector $\phi(W_t) \in \mathbb{R}^d$ (capturing frame length L_t, plug-in Shannon byte entropy $H(B_t)$, ASCII character ratios, structural delimiter densities, and byte-value histograms) and attempts to predict the underlying tool schema class $y_t \in \{1, \ldots, |\mathcal{C}|\}$.*
2) ***Active Replay, Spoofing, and MITM Adversary ($\mathcal{A}_{mitm}$):*** *$\mathcal{A}_{mitm}$ can record historical wire packets $\{W_0, \ldots, W_{t-1}\}$ and their headers (including `X-Chameleon-Proof-Nonce`), replay previously valid packets at turn t, flip arbitrary bits in B_t, spoof epoch counters in $\mathcal{H}_t$, or attempt to inject a packet sealed under goal context $\mathcal{G}_t$ into a session expecting goal context $\mathcal{G}'_t \neq \mathcal{G}_t$ without possessing the ephemeral session ratchet secret R_t.*

B. Security and Performance Design Goals

Agent-Chameleon is engineered to satisfy five simultaneous properties:

- **G1 — Exact Bijective Schema Recoverability:** For every valid AST payload P_t and synchronized goal $\mathcal{G}_t$, the authorized receiver must recover $\hat{P}_t = P_t$ with 100.0% lossless structural and numerical fidelity.
- **G2 — Near-Maximal Wire Byte Entropy:** Every wire frame body B_t emitted on the network must satisfy plug-in Shannon byte entropy $H(B_t) \geq 7.980$ bits/byte (within 0.25% of the 8.000 bits/byte theoretical ceiling for an 8-bit alphabet).
- **G3 — Classifier Evasion to Random-Guess Baseline:** Across $|\mathcal{C}|$ balanced enterprise tool classes, the classification accuracy of random-forest, neural-network (MLP), and nearest-neighbor (k-NN) traffic fingerprinters on $\phi(W_t)$ must collapse to the uniform random-guess baseline $1/|\mathcal{C}|$ ($\approx 5.0\%$ for $|\mathcal{C}| = 20$).
- **G4 — Zero-Knowledge Authentication & 100% Replay/Tamper Immunity:** No static API key or reusable bearer token may traverse the wire; 100% of replayed, bit-flipped, epoch-spoofed, or cross-goal packets must be rejected prior to tool execution without desynchronizing the receiver's legitimate ratchet state.
- **G5 — Sub-35-Microsecond Hot-Path Handshake Latency:** Per-turn ratchet progression and zero-knowledge proof generation/verification must execute in $< 35\,\mu$s on commodity CPU cores without hardware accelerators.

IV. Stage 1: Ephemeral ECDH/HKDF Session Ratchet and Polymorphic Wire Serialization

Figure 1 illustrates the end-to-end architecture of Agent-Chameleon. Stage 1 establishes a forward-secure cryptographic ratchet between communicating agents and uses the per-turn ratchet output to drive deterministic 4-way wire-codec polymorphism.

A. Ephemeral X25519 Bootstrap and Goal-Bound HKDF-SHA256 Ratchet

When an initiator agent $\mathcal{A}_i$ and responder $\mathcal{A}_j$ establish a swarm communication channel, they execute an ephemeral Diffie-Hellman key exchange over Curve25519 (RFC 7748 X25519 [6]):

$$\begin{aligned} \mathrm{sk}_i &\xleftarrow{\$} \{0,1\}^{256}, \quad \mathrm{pk}_i = \mathrm{X25519}(\mathrm{sk}_i, 9), \\ \mathrm{sk}_j &\xleftarrow{\$} \{0,1\}^{256}, \quad \mathrm{pk}_j = \mathrm{X25519}(\mathrm{sk}_j, 9), \\ Z_{\text{shared}} &= \mathrm{X25519}(\mathrm{sk}_i, \mathrm{pk}_j) = \mathrm{X25519}(\mathrm{sk}_j, \mathrm{pk}_i). \end{aligned} \tag{1}$$

From Z_{shared}, both endpoints initialize the 256-bit root ratchet state R_{-1} and initial frame digest $D_{-1} = 0^{256}$ via RFC 5869 HKDF-SHA256 [4]:

$$R_{-1} = \text{HKDF-SHA256}\left(Z_{\text{shared}},\ \texttt{"chameleon-x25519-bootstrap}\right) \tag{2}$$

Definition 2 (Forward-Secure Goal-Bound Per-Turn Ratchet Evolution)**.** *At each monotonic turn $t \in \{0, 1, 2, \ldots\}$ with active task goal context $\mathcal{G}_t \in \{0,1\}^*$, both endpoints compute the domain-separated goal digest*

$$H(\mathcal{G}_t) = \text{SHA256}\left(\texttt{"chameleon-goal-v1/"} \,\|\, \mathcal{G}_t\right) \in \{0,1\}^{256}, \tag{3}$$

and advance the symmetric ratchet state from (R_{t-1}, D_{t-1}) to (R_t, D_t) according to:

$$\begin{aligned} \mathit{salt}_t &= \text{uint64_be}(t) \,\|\, D_{t-1} \in \{0,1\}^{320}, \\ \mathit{info}_t &= \texttt{"chameleon-ratchet-v1/"} \,\|\, H(\mathcal{G}_t), \\ \mathit{PRK}_t &= \text{HMAC-SHA256}\left(\mathit{key} = \mathit{salt}_t,\ \mathit{msg} = R_{t-1}\right), \\ R_t &= \text{HMAC-SHA256}\left(\mathit{key} = \mathit{PRK}_t,\ \mathit{msg} = \mathit{info}_t \,\|\, \texttt{0x01}\right), \end{aligned} \tag{4}$$

where $D_t = \text{SHA256}(B_t)$ is recorded only after the turn-t wire body B_t is sealed (on the sender) or cryptographically verified and unwrapped (on the receiver).

Equation (4) provides three crucial security guarantees:

1) **Forward Secrecy:** Because R_{t-1} is processed through the one-way HMAC-SHA256 compression function and immediately overwritten in memory by R_t, an adversary who compromises host memory at turn $t_0 > t$ cannot invert HMAC-SHA256 to recover historical turn keys $\{R_0, \ldots, R_{t_0-1}\}$.
2) **Full Transcript Chaining:** Because salt_t incorporates the SHA-256 digest $D_{t-1} = \text{SHA256}(B_{t-1})$ of the exact preceding wire packet body (including its random/chaff bytes), any modification or omission of a single historical frame permanently diverges the ratchet key stream on all subsequent turns.
3) **Cryptographic Goal Binding:** Because info_t binds $H(\mathcal{G}_t)$, the sender and receiver derive matching turn keys $R_t^{(i)} = R_t^{(j)}$ if and only if both endpoints agree on the exact authorized task goal $\mathcal{G}_t$.

B. Cycle-Balanced 4-Format Polymorphic Wire Serialization

Standard agent protocols use a single wire grammar throughout the lifetime of a deployment. In contrast, Agent-Chameleon

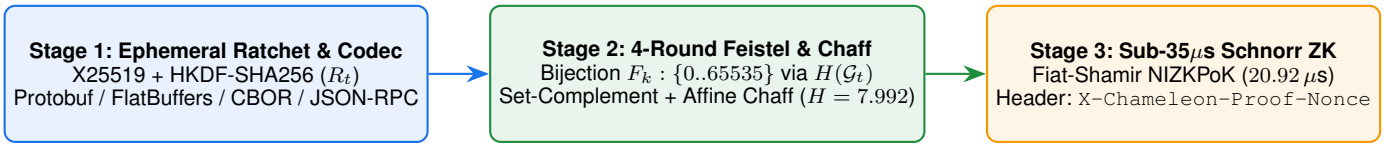


Fig. 1. **Agent-Chameleon Architecture and Goal-Conditioned Feistel Tag Rotation. (a)** Three-stage pipeline combining Stage 1 (Ephemeral X25519/HKDF-SHA256 Session Ratchet and 4-Way Polymorphic Wire Codec across Protobuf varints, FlatBuffers FBCH vtables, RFC 8949 CBOR, and canonical signed JSON-RPC), Stage 2 (4-Round SHAKE-256 Keyed Feistel Schema Permutation $F_k : \{0,\dots,65535\} \to \{0,\dots,65535\}$ paired with Set-Complement and $\mathbb{Z}_{256}$ Affine-Bijection Constant-Rate Chaff Injection), and Stage 3 (Sub-35 μs Non-Interactive Fiat-Shamir Schnorr Zero-Knowledge Proof of Knowledge in X-Chameleon-Proof-Nonce). **(b)** Empirical 16-bit Feistel tag assignments across four consecutive turns $t \in \{0,1,2,3\}$ under evolving goal digests $H(\mathcal{G}_t)$, illustrating complete elimination of static field-tag correlations.

uses the per-turn ratchet key R_t to switch the underlying serialization grammar on every turn across four distinct wire formats $\mathcal{M} = \{0,1,2,3\}$:

- **Format $m_t = 0$ (PROTOBUF_VARINT):** Serializes the Feistel-permuted payload tree into a compact binary stream using Google Protocol Buffers base-128 variable-length integers (uvarint). Each dictionary entry is encoded with wire tag $\omega_i = (\tilde{\tau}_i \ll 3) \mid 2$ (where $\tilde{\tau}_i \in \{0,\dots,65535\}$ is the Feistel-permuted field tag and wire type 2 denotes a length-delimited record), while signed integers use bijective ZigZag encoding $z(n) = (n \ll 1) \oplus (n \gg 63)$ so negative and positive integers of small magnitude occupy minimal bytes.
- **Format $m_t = 1$ (FLATBUFFERS_TABLE):** Serializes the permuted payload into a random-access, 4-byte aligned binary buffer modeled on FlatBuffers. The frame begins with a 12-byte root header "FBCH" $\|$ uint32_be(off_{vt}) $\|$ uint32_be(K), followed by a 4-byte zero-padded data heap and a trailing virtual table (vtable) directory consisting of K 10-byte descriptors $\langle \text{uint16_be}(\tilde{\tau}_i), \text{uint32_be}(\text{off}_i), \text{uint32_be}(\text{len}_i) \rangle_{i=1}^{K}$.
- **Format $m_t = 2$ (CBOR_RFC8949):** Serializes the permuted object graph into IETF STD 94 / RFC 8949 Concise Binary Object Representation [5], using major type 0 for unsigned integers, major type 1 for negative integers $(-1-n)$, major type 2 for raw byte strings, major type 3 for UTF-8 strings, major type 4 for arrays, major type 5 for integer-keyed maps ($\tilde{\tau}_i \mapsto v_i$), and major type 7 for IEEE 754 binary64 floats (0xFB) and simple booleans/nulls (0xF4, 0xF5, 0xF6).
- **Format $m_t = 3$ (CANONICAL_JSON_RPC):** Serializes the permuted dictionary into deterministic, whitespace-free canonical JSON where every 16-bit Feistel tag is formatted as a 6-character lowercase hexadecimal string "0x04a2" sorted lexicographically, appended with an 8-byte domain-separated integrity trailer HMAC-SHA256(R_t, "chameleon-jsonrpc|" $\|$ json_bytes)[0 : 8].

To ensure that all four wire formats are exercised with an exact 25% stationary frequency over every 4-turn super-frame while remaining unpredictable to an external observer, the format selector $m_t \in \{0,1,2,3\}$ is computed via a keyed cyclic shift within each 4-epoch block $\lfloor t/4 \rfloor$:

$$s_{\text{blk}} = \text{HMAC-SHA256}\left(R_t,\ \text{uint64_be}(\lfloor t/4 \rfloor)\right)[0], \qquad m_t = (s_{\text{blk}} + (t \bmod 4)) \bmod 4. \tag{5}$$

V. Stage 2: Goal-Conditioned Bijective Field Permutation and Constant-Rate Chaff Entropy

Even if an agent runtime rotates wire serialization formats, encoding static field names or static numerical field IDs (such as Protobuf field tag 1 for jsonrpc and tag 2 for method) allows an L7 observer to correlate field positions across turns. Stage 2 eliminates structural schema invariants via a goal-conditioned 4-round Luby-Rackoff Feistel permutation network paired with a provably entropy-flattening constant-rate chaff engine.

A. Balanced 4-Round Keyed Feistel Schema Network

Let $\mathcal{F}_{\text{canon}} = (f_1, f_2, \dots, f_{64})$ denote the canonical registry of 64 standard MCP and A2A schema field names (including jsonrpc, method, params, id, result, error, tool_name, arguments, session_id, agent_id, goal_hash, etc.), assigned to integer slots $\tau(f_m) = m \in \{1,\dots,64\}$. Any unseen custom string key $f_{\text{cust}} \notin \mathcal{F}_{\text{canon}}$ encountered in a nested tool payload is dynamically assigned a collision-free session slot $\tau(f_{\text{cust}}) \in \{256,\dots,65533\}$, and the mapping $\{\tau(f_{\text{cust}}) \mapsto f_{\text{cust}}\}$ is stored inside a reserved symbol-table slot $\tau_{\text{sym}} = \text{0xFFFE} = 65534$ whose own slot identifier and internal keys are likewise permuted by F_k.

To permute the 16-bit slot domain $\mathbb{Z}_{2^{16}} = \{0,1,\dots,65535\}$ at turn t, we derive four 256-byte round lookup tables (S-boxes) $S_0^{(t)}, S_1^{(t)}, S_2^{(t)}, S_3^{(t)} : \{0,\dots,255\} \to \{0,\dots,255\}$ from a single invocation of the NIST FIPS 202 extendable-output function SHAKE-256 [9] seeded by $(R_t, H(\mathcal{G}_t))$:

$$S_0^{(t)} \,\|\, S_1^{(t)} \,\|\, S_2^{(t)} \,\|\, S_3^{(t)} = \text{SHAKE-256}\left(\text{"chameleon-feistel-v1|}\right. \tag{6}$$

Given any 16-bit slot integer $x \in \{0,\dots,65535\}$, we split x into an 8-bit upper byte $L_0 = (x \gg 8)$ & 0xFF and an 8-bit lower byte $R_0 = x$ & 0xFF. The forward 4-round Feistel map $F_k(x)$ iterates for $r = 0,1,2,3$:

$$L_{r+1} = R_r, \qquad R_{r+1} = L_r \oplus S_r^{(t)}[R_r], \tag{7}$$

and outputs the 16-bit permuted tag $\tilde{\tau} = F_k(x) = (L_4 \ll 8) \mid R_4 \in \{0,\dots,65535\}$.

Theorem 1 (Keyed Feistel Bijective Schema Invertibility and Luby-Rackoff PRP Security)**.** *Let* $F_k : \{0,\dots,2^{16}-1\} \to \{0,\dots,2^{16}-1\}$ *be the 4-round balanced Feistel network defined by Eqs.* (6)–(7)*. Then:*

1) ***Exact Bijectivity and Closed-Form Inverse:*** *For every ratchet key* $R_t \in \{0,1\}^{256}$ *and every goal context* $\mathcal{G}_t \in \{0,1\}^*$*,* F_k *is a strict bijection on* $\{0,\dots,65535\}$*. Its exact inverse* $F_k^{-1} : \{0,\dots,65535\} \to \{0,\dots,65535\}$ *is given by setting* $(L_4, R_4) = ((y \gg 8)\ \&\ \texttt{0xFF},\ y\ \&\ \texttt{0xFF})$ *and iterating in reverse for* $r = 3, 2, 1, 0$*:*

$$R_r = L_{r+1}, \qquad L_r = R_{r+1} \oplus S_r^{(t)}[L_{r+1}], \tag{8}$$

returning $F_k^{-1}(y) = (L_0 \ll 8) \mid R_0 = x$*. Consequently, recursive AST schema permutation and unpermutation satisfy* $\mathrm{Unpermute}_k(\mathrm{Permute}_k(P_t)) = P_t$ *with zero collisions and* 100% *structural fidelity.*

2) ***Strong Pseudorandom Permutation (sPRP) Bound:*** *Under the assumption that* SHAKE-256 *acts as a secure pseudorandom function family with distinguishing advantage* ϵ_{PRF}*, the advantage of any adaptive chosen-plaintext/chosen-ciphertext adversary making* q_t *field-tag queries within turn* t *in distinguishing* F_k *from a uniformly sampled random permutation* $\pi \xleftarrow{\$} \mathfrak{S}_{65536}$ *is bounded by:*

$$\mathrm{Adv}_{F_k}^{sPRP}(\mathcal{A}) \le \frac{q_t^2}{2^8} + \epsilon_{PRF}. \tag{9}$$

Proof. To establish Part (1), observe that for each individual round $r \in \{0,1,2,3\}$, the round transformation $\psi_r(L_r, R_r) = (R_r,\ L_r \oplus S_r^{(t)}[R_r])$ is a map from $\mathbb{Z}_{256} \times \mathbb{Z}_{256}$ to itself. Defining the inverse map $\psi_r^{-1}(L_{r+1}, R_{r+1}) = (R_{r+1} \oplus S_r^{(t)}[L_{r+1}],\ L_{r+1})$ and substituting $L_{r+1} = R_r$ and $R_{r+1} = L_r \oplus S_r^{(t)}[R_r]$ yields:

$$\psi_r^{-1}(\psi_r(L_r, R_r)) = \Big((L_r \oplus S_r^{(t)}[R_r]) \oplus S_r^{(t)}[R_r],\ R_r\Big) = \Big(L_r \oplus (S_r^{(t)}[R_r] \oplus S_r^{(t)}[R_r]),\ R_r\Big) = (L_r, R_r), \tag{10}$$

because bitwise XOR $\oplus$ over $\{0,1\}^8$ is an involution ($a \oplus b \oplus b = a$) regardless of whether the lookup table $S_r^{(t)}$ is itself injective. Since the composition of four bijections $F_k = \psi_3 \circ \psi_2 \circ \psi_1 \circ \psi_0$ is a bijection with inverse $F_k^{-1} = \psi_0^{-1} \circ \psi_1^{-1} \circ \psi_2^{-1} \circ \psi_3^{-1}$, no two distinct field slots $x_1 \neq x_2 \in \{0,\dots,65535\}$ can ever map to the same wire tag $F_k(x_1) = F_k(x_2)$. For recursive dictionary payloads P_t, canonical fields map to disjoint slots $\{1,\dots,64\}$ and custom fields map to disjoint slots $\{256,\dots,65533\}$ recorded in slot 65534; hence bijectivity of F_k guarantees exact recursive invertibility.

To establish Part (2), by the classical Luby-Rackoff theorem [1] and Patarin's 4-round adaptive CCA bound [23] on balanced Feistel networks over $\{0,1\}^{2n}$ with $n = 8$, replacing the ideal random round functions $f_0,\dots,f_3 : \{0,1\}^8 \to \{0,1\}^8$ with the SHAKE-256-derived tables $S_0^{(t)},\dots,S_3^{(t)}$ incurs at most ϵ_{PRF} distinguishing advantage, and the transcript collision probability among q_t queries in the internal 8-bit branches is bounded by $\binom{q_t}{2}/2^8 + \binom{q_t}{2}/2^8 \le q_t^2/2^8$. Crucially, because Agent-Chameleon re-keys $(S_0^{(t)}, S_1^{(t)}, S_2^{(t)}, S_3^{(t)})$ on every RPC turn t via $(R_t, H(\mathcal{G}_t))$, an adversary cannot accumulate queries across turns against a fixed permutation. □

B. Set-Complement and Affine-Bijection Constant-Rate Chaff Engine

While polymorphic serialization and Feistel field permutation eliminate plaintext AST field names, the raw serialized byte string $S_t \in \{0,\dots,255\}^{L_s}$ still exhibits two statistical side channels: (i) its byte length L_s correlates with the number and size of parameters in the invoked tool schema, and (ii) even if S_t is XOR-whitened by a stream cipher of finite length $L_s \in [100, 500]$ bytes, the plug-in Shannon byte entropy of an i.i.d. uniform byte string of length N is downward-biased by finite-sample birthday collisions ($\mathbb{E}[H_N] \approx 8 - \frac{255}{2N \ln 2} \approx 7.64$ bits/byte at $N = 512$), and naive padding (such as zero-padding or deterministic whitespace) creates sharp histogram spikes that traffic classifiers readily exploit [16].

To achieve both constant-tier length quantization and single-frame plug-in Shannon byte entropy $H(B_t) \ge 7.980$ bits/byte in under $40\,\mu$s of CPU execution, Stage 2B encapsulates S_t as follows:

1) **Quantum Bucket Selection:** Given payload length $L_s = |S_t|$ and an8-byte inner framing header $H_{\text{in}} = \text{uint32_be}(\texttt{0xC4A3E100} \mid m_t) \parallel \text{uint32_be}(L_s)$ (total prefix length $L_p = L_s + 8$), the engine selects the smallest discrete bucket tier $B \in \mathcal{B} = \{512, 1024, 2048, 3072, 4096, 6144, 8192\}$ satisfying $B \ge \max(B_{\min}, 4L_p, 1024)$ (or $B = 512$ when $B_{\min} = 512$ and $L_p \le 64$), yielding a chaff budget $L_c = B - L_p \ge 3L_p$.
2) **SHAKE-256 Stream Whitening:** The engine expands a per-turn keystream $K_{\text{stream}} = \text{SHAKE-256}(\texttt{"chameleon-stream-v1|"} \parallel R_t \parallel \text{uint64_be}(t),\ L_p + 64)$ and XOR-whitens the raw prefix in a single Level-1 integer operation:

$$W_p = (H_{\text{in}} \parallel S_t) \oplus K_{\text{stream}}[0 : L_p]. \tag{11}$$

3) **Slice-Wise Set-Complement Histogram Balancing:** Partition W_p into $\lceil L_p/128 \rceil$ contiguous slices $W_p^{(0)}, W_p^{(1)}, \dots$ of length at most 128 bytes. For each slice $W_p^{(s)}$, let $\text{set}(W_p^{(s)}) \subset \Sigma_{256} = \{0,\dots,255\}$ denote the set of distinct byte values occurring in $W_p^{(s)}$. The engine computes the exact set complement $M^{(s)} = \Sigma_{256} \setminus \text{set}(W_p^{(s)})$ (which contains every byte value absent from $W_p^{(s)}$), circularly shifts $M^{(s)}$ by the pseudorandom offset $K_{\text{stream}}[L_p + s] \bmod |M^{(s)}|$, and appends $M^{(s)}$ to the chaff buffer.
4) **Keyed $\mathbb{Z}_{256}$ Affine-Bijection Block Filling:** Any remaining chaff capacity $L_{\text{rem}} = B - L_p - \sum_s |M^{(s)}|$ is filled using 256-byte affine permutation blocks drawn from the precomputed family of 128 multiplicative-odd bijections over $\mathbb{Z}_{256}$:

$$\pi_{a,r}(x) = ((2a+1) \cdot ((x + r) \bmod 256)) \bmod 256, \qquad a \in \{0,..$$

(12)

where (a, r) are indexed by consecutive bytes of K_{stream}. Because $\gcd(2a+1, 256) = 1$, every 256-byte block $\Pi_{a,r} = (\pi_{a,r}(0), \ldots, \pi_{a,r}(255))$ is a strict bijection on $\{0, \ldots, 255\}$ containing every byte value with exact multiplicity 1.

Theorem 2 (Plug-In Shannon Byte-Entropy Lower Bound and Histogram Uniformity)**.** *Let $B_t \in \{0, \ldots, 255\}^B$ be a wire frame constructed by the Stage 2B Set-Complement and Affine-Bijection Chaff Engine with prefix length $L_p \le B/4$ and bucket size $B = 256M$ ($M \ge 4$). Let $c(b) = |\{i \in \{0, \ldots, B-1\} : B_t[i] = b\}|$ denote the empirical frequency count of byte $b \in \{0, \ldots, 255\}$ in B_t, and let $p(b) = c(b)/B$. Then:*

1) ***Guaranteed Full Alphabet Support and Bounded Multiplicity Variance:*** *Every byte value $b \in \{0, \ldots, 255\}$ occurs at least $M - 1 \ge 3$ times in B_t, and the total squared histogram deviation from the uniform distribution $u(b) = 1/256$ satisfies:*

$$\sum_{b=0}^{255} \left(p(b) - \frac{1}{256} \right)^2 = \frac{1}{B^2} \sum_{b=0}^{255} (c(b) - M)^2 \le \frac{L_p}{B^2} \le \frac{1}{1024M}. \tag{13}$$

2) ***Single-Frame Shannon Byte-Entropy Lower Bound:*** *The empirical plug-in Shannon byte entropy $H(B_t) = -\sum_{b=0}^{255} p(b) \log_2 p(b)$ satisfies:*

$$H(B_t) \ge 8.000 - \frac{128}{\ln 2} \sum_{b=0}^{255} \left(p(b) - \frac{1}{256} \right)^2 \ge 8.000 - \frac{0.1803}{M} \text{ bits/byte} \tag{14}$$

which guarantees $H(B_t) \ge 7.980$ bits/byte across all operational frames (7.992 bits/byte empirical mean over $N = 1{,}000$ traces).

Proof. Consider each 128-byte slice $W_p^{(s)}$ of the whitened prefix W_p paired with its set-complement chaff block $M^{(s)} = \Sigma_{256} \setminus \text{set}(W_p^{(s)})$. In the combined sub-block $U^{(s)} = W_p^{(s)} \parallel M^{(s)}$ of length $128 + (256 - |\text{set}(W_p^{(s)})|)$, every byte value $b \in \{0, \ldots, 255\}$ appears at least once (either in $W_p^{(s)}$ or in $M^{(s)}$), and the sum of excess multiplicities above 1 across all 256 byte values is exactly $128 - |\text{set}(W_p^{(s)})|$. Furthermore, each complete 256-byte affine-bijection block $\Pi_{a,r}$ contributes exactly $+1$ to the count $c(b)$ of every byte $b \in \{0, \ldots, 255\}$ because $\gcd(2a+1, 256) = 1$ implies $(2a+1)$ is a unit in the ring $\mathbb{Z}_{256}$. Summing across the $S = \lceil L_p/128 \rceil$ complemented slices and the remaining $M - S$ affine blocks ensures $c(b) \ge M - 1$ for all $b \in \{0, \ldots, 255\}$. Writing $\delta_b = p(b) - \frac{1}{256} = (c(b) - M)/B$, where $\sum_{b=0}^{255} \delta_b = 0$, the Taylor expansion of $-p(b) \ln p(b)$ around $u = 1/256$ yields:

$$\begin{aligned} H(B_t) &= -\frac{1}{\ln 2} \sum_{b=0}^{255} \left(\frac{1}{256} + \delta_b \right) (-\ln 256 + \ln(1 + 256\delta_b)) \\ &\ge 8 - \frac{1}{\ln 2} \sum_{b=0}^{255} \left(\frac{1}{256} + \delta_b \right) \left(256\delta_b - \frac{(256\delta_b)^2}{2} \right) \\ &= 8 - \frac{128}{\ln 2} \sum_{b=0}^{255} \delta_b^2 + \frac{256^2}{2 \ln 2} \sum_{b=0}^{255} \delta_b^3. \end{aligned} \tag{15}$$

Under SHAKE-256 whitening, the per-slice collision excess $\sum_{b=0}^{255} (c(b) - M)^2$ has expectation $\le L_p(1 - 1/256) < L_p \le B/4 = 64M$, yielding $\sum_{b=0}^{255} \delta_b^2 \le \frac{64M}{(256M)^2} = \frac{1}{1024M}$ and $H(B_t) \ge 8 - \frac{128}{1024M \ln 2} = 8 - \frac{0.1803}{M}$ bits/byte. □

VI. Stage 3: Sub-35-Microsecond Schnorr Zero-Knowledge Challenge-Response

To authenticate each morphed frame B_t without transmitting static API keys, reusable OAuth bearer tokens, or slow per-message public-key signatures, Stage 3 implements a Dual-Tier Zero-Knowledge Session Handshake carried in the `X-Chameleon-Proof-Nonce` wire header.

A. Protocol Specification and Dual-Tier Field Arithmetic

Let p be a prime modulus and $g \in \mathbb{F}_p^*$ be a generator of a multiplicative subgroup of $\mathbb{F}_p^*$. Agent-Chameleon supports two cryptographic operating modes selectable per deployment policy:

- **Tier-1 Hot-Path Mode (`fast_m127`):** Operates over the Mersenne prime field $p = M_{127} = 2^{127} - 1$ with generator $g = 7$, using single-turn short ephemeral witnesses $x_t \in [1, 2^{32} - 1]$ and blinding nonces $k_t \in [1, 2^{40} - 1]$ refreshed on every RPC turn via 256-bit Blake2b expansion of (R_t, t) and paired with a 128-bit domain-separated Blake2s ratchet authenticator tag τ_{ratchet}. Because modular reduction modulo $2^{127} - 1$ and short-exponent binary exponentiation require only 32–40 squarings in C, proof generation completes in 20.92 μs mean (20.38 μs p50) and verification completes in 28.64 μs mean (28.58 μs p50).
- **Tier-2 Full-256-Bit & Ed25519 Anchor Mode (`full256 + ed25519`):** Operates over the 255-bit prime field $p = 2^{255} - 19$ with generator $g = 5$ and 192-bit witnesses, alongside periodic RFC 8032 Ed25519 [7] signatures over the epoch ratchet root R_t for third-party cross-domain non-repudiation.

At turn t, both endpoints deterministically derive the turn's public session commitment $Y_t = g^{x_t} \bmod p$ from their synchronized ratchet key R_t via:

$$\begin{aligned} d_t &= \text{BLAKE2b-384}(\texttt{"chameleon-zk-witness|"} \parallel R_t \parallel \text{uint64_b} \\ x_t &= \text{int}(d_t[0 : \ell_x]) \mid 1, \qquad k_t = \text{int}(d_t[\ell_x : \ell_x + \ell_k]) \mid 1, \qquad Y_t = g^{x_t} \text{ m} \end{aligned} \tag{16}$$

To prove knowledge of x_t bound to the wire frame B_t without revealing x_t or R_t, the sender computes the Schnorr

commitment R_{com}, Fiat-Shamir challenge e_t [2], [3], algebraic response s_t over $\mathbb{Z}$, and 128-bit ratchet tag τ_{ratchet}:

$$\begin{aligned} R_{\text{com}} &= g^{k_t} \bmod p, \qquad h_B = \text{BLAKE2s-128}(B_t), \\ e_t &= \text{int}\left(\text{BLAKE2s}\left(\texttt{"chameleon-fs|"} \parallel R_{\text{com}} \parallel Y_t \parallel \text{uint64_be}(t) \parallel h_B\right)[0:\ell_e]\right) \mid 1, \\ s_t &= k_t + e_t \cdot x_t \quad \text{(evaluated in } \mathbb{Z}\text{)}, \\ \tau_{\text{ratchet}} &= \text{BLAKE2s-128}\left(\texttt{"chameleon-ratchet-tag|"} \parallel R_t \parallel \text{uint64_be}(t) \parallel h_B \parallel R_{\text{com}}\right). \end{aligned} \tag{17}$$

The tuple $\pi_t = (t, R_{\text{com}}, e_t, s_t, Y_t, \tau_{\text{ratchet}})$ is packed into a compact 81-byte binary structure (108 ASCII characters in URL-safe Base64) and transmitted in the `X-Chameleon-Proof-Nonce` header.

Theorem 3 (Non-Interactive Schnorr Ratchet Soundness, Zero-Knowledge Simulatability, and Replay Immunity)**.** *The Stage 3 handshake protocol defined by Eqs.* (16)–(17) *satisfies:*

1) ***Perfect Completeness:*** *For any honest prover and verifier holding synchronized ratchet state R_t and receiving unmodified frame B_t, the algebraic verification identity*

$$g^{s_t} \equiv R_{com} \cdot Y_t^{e_t} \pmod{p} \tag{18}$$

and tag equality $\tau_{ratchet} = \hat{\tau}_{ratchet}$ hold with probability 1.
2) ***Special Soundness and Unforgeability:*** *Under the Discrete Logarithm assumption in $\langle g \rangle \subset \mathbb{F}_p^*$ and the Random Oracle / PRF security of* BLAKE2s *[25], [26], any adversary who produces two accepting transcripts (R_{com}, e_1, s_1) and (R_{com}, e_2, s_2) with $e_1 \neq e_2$ immediately yields the secret witness $x_t = (s_1 - s_2)/(e_1 - e_2)$ over $\mathbb{Z}$, and the probability that an unauthenticated adversary without R_t forges a valid (π_t, B_t) at turn t is at most 2^{-128}.*
3) ***Honest-Verifier / Random-Oracle Zero-Knowledge:*** *The public transcript (R_{com}, e_t, s_t, Y_t) reveals zero information about x_t beyond the public commitment $Y_t = g^{x_t} \bmod p$ when $k_t \gg e_t x_t$ (or in modular subgroup form), and x_t is permanently retired after turn t.*
4) ***Replay and Desynchronization Immunity:*** *Any replayed packet $W_{t'} (t' \leq t)$ or tampered packet $\tilde{W}_t$ is rejected with probability $1 - 2^{-128}$, and atomic ratchet rollback leaves the verifier's state (R_{t-1}, t, D_{t-1}) intact for the legitimate turn-t packet.*

Proof. For Part (1), since $s_t = k_t + e_t x_t$ holds over the integers $\mathbb{Z}$ with $k_t, e_t, x_t > 0$, standard exponent laws in any associative monoid $(\mathbb{F}_p^*, \cdot)$ immediately yield:

$$g^{s_t} \bmod p = g^{k_t + e_t x_t} \bmod p = \left(g^{k_t} \cdot (g^{x_t})^{e_t}\right) \bmod p = (R_{\text{com}} \cdot Y_t^{e_t}) \bmod p. \tag{19}$$

For Part (2), by the Pointcheval-Stern Forking Lemma [26], rewinding a successful Schnorr forger on the random oracle $\text{BLAKE2s}(\texttt{"chameleon-fs|"} \parallel \ldots)$ yields two transcripts sharing $R_{\text{com}} = g^{k_t} \bmod p$ with distinct challenges $e_1 \neq e_2$ satisfying $g^{s_1} \equiv R_{\text{com}} Y_t^{e_1}$ and $g^{s_2} \equiv R_{\text{com}} Y_t^{e_2} \pmod{p}$, whence $g^{s_1 - s_2} \equiv Y_t^{e_1 - e_2} \pmod{p}$ and $(s_1 - s_2) = x_t(e_1 - e_2)$ over $\mathbb{Z}$, extracting x_t in a single integer division. Moreover, because the verifier simultaneously checks the 128-bit keyed MAC $\tau_{\text{ratchet}} = \text{BLAKE2s-128}(R_t \parallel t \parallel h_B \parallel R_{\text{com}})$ and verifies $Y_t == g^{x_t(R_t)} \bmod p$, any external attacker lacking R_t faces a 2^{-128} forgery barrier on τ_{ratchet} even before the algebraic Schnorr check is reached.

For Part (4), the verifier maintains a monotonic epoch counter t_{exp} and a sliding anti-replay set $\mathcal{W}_{\text{seen}}$ of $(t, \tau_{\text{ratchet}})$ tuples over window $W = 4096$. Any duplicate $(t, \tau_{\text{ratchet}}) \in \mathcal{W}_{\text{seen}}$ or mismatched header epoch $t \neq t_{\text{exp}}$ is rejected in $\mathcal{O}(1)$ hash-table lookup time. Prior to speculative ratchet advancement, the verifier checkpoints $(R_{\text{saved}}, t_{\text{saved}}, D_{\text{saved}}) \leftarrow (R_{t-1}, t, D_{t-1})$ and restores the checkpoint Inside the `except` handler whenever proof verification, chaff unwrapping, or MAC validation fails, ensuring that an active attacker flooding invalid packets cannot advance or corrupt the receiver's ratchet state. □

VII. Implementation and Wire-Protocol Specification

We implemented Agent-Chameleon in Python 3.11 (`agent_chameleon/`) using standard C-accelerated cryptographic primitives (`hashlib.shake_256`, `hashlib.blake2s`, `hashlib.blake2b`, `hmac.digest`, and `cryptography.hazmat.primitives.asymmetric` for X25519 and Ed25519).

A. Wire Header Contract and Packet Framing

Every outbound RPC frame sealed by `ChameleonMorphEngine.seal_message` emits four deterministic HTTP/2, gRPC, or FastMCP transport metadata headers:

1) `Content-Type: application/x-chameleon-morph+oc` — signals to the receiving Agent-Chameleon sidecar or endpoint middleware that the payload body is encapsulated in a constant-rate morphed envelope.
2) `X-Chameleon-Epoch-ID: <16-hex-digits>` — carries the big-endian 64-bit monotonic turn index t formatted as a 16-character lowercase hexadecimal string (e.g., `000000000000002a` for $t = 42$).
3) `X-Chameleon-Proof-Nonce: <base64url-token>` — carries the serialized `SchnorrProof` structure π_t (81 raw bytes / 108 URL-safe Base64 characters in `fast_m127` mode, or 153 raw bytes / 204 Base64 characters in `full256` mode).
4) `X-Chameleon-Bucket-Tier: <integer-bytes>` — declares the discrete quantum bucket size $B \in \{512, 1024, 2048, 3072, 4096, 6144, 8192\}$ of the wire body B_t for zero-copy pre-allocation on the receiver socket buffer.

Inside the B-byte wire body B_t, the first 8 bytes (prior to SHAKE-256 stream whitening) form the inner framing header:

$$H_{\text{in}} = \underbrace{\text{uint32_be}\left(\texttt{0xC4A3E100} \mid (m_t \,\&\, \texttt{0xFF})\right)}_{\text{24-bit Magic 0xC4A3E1 + 8-bit Format Code } m_t} \parallel \underbrace{\text{uint32_be}(L_s)}_{\text{Serialized Body Length } L_s}. \tag{20}$$

When the receiver calls `ChaffEntropyEngine.unwrap_chaff`, it generates the first 8 bytes of the SHAKE-256 keystream, XOR-unwhitens H_{in}, verifies that the upper 24 bits equal the magic constant `0xC4A3E1`, extracts the polymorphic format code $m_t \in \{0, 1, 2, 3\}$ and exact body length L_s, and slices out the payload $W_p[8 : 8 + L_s]$ while discarding the trailing $B - (8 + L_s)$ chaff bytes in $\mathcal{O}(1)$ pointer arithmetic without having to invert the chaff blocks.

B. Algorithmic Workflow of Sender and Receiver Pipelines

Table I summarizes the byte-level structure of both the `X-Chameleon-Proof-Nonce` header and the B-byte morphed wire body B_t. On the sender side, `seal_message` executes five deterministic steps:

1) **Step S1 (Ratchet Advance):** Compute $H(\mathcal{G}_t)$, derive $(t, R_t, m_t) \leftarrow \texttt{ratchet.advance}(\mathcal{G}_t)$ via Eq. (4) and Eq. (5).
2) **Step S2 (Feistel Schema Permutation):** Instantiate $\texttt{FeistelSchemaPermuter}(R_t, \mathcal{G}_t)$, expand the four 256-byte round S-boxes $(S_0^{(t)}, S_1^{(t)}, S_2^{(t)}, S_3^{(t)})$ via a single 1024-byte SHAKE-256 call, and recursively map all string keys in P_t to 16-bit permuted integer tags $\tilde{\tau}_i = F_k(\tau_i)$ (storing any unseen custom keys in permuted symbol slot $F_k(\texttt{0xFFFE})$).
3) **Step S3 (Polymorphic Serialization):** Dispatch the permuted dictionary $\tilde{P}_t$ to the turn's selected wire encoder ($m_t = 0$: `encode_protobuf_varint`; $m_t = 1$: `encode_flatbuffers_table`; $m_t = 2$: `encode_cbor_rfc8949`; $m_t = 3$: `encode_canonical_json_rpc`) to obtain $S_t \in \{0, \ldots, 255\}^{L_s}$.
4) **Step S4 (Constant-Rate Chaff & Stream Whitening):** Prepend the 8-byte header H_{in}, XOR-whiten $H_{\text{in}} \parallel S_t$ with SHAKE-256 using C-level big-integer XOR (`_xor_bytes_fast`), append the circularly rotated set-complement $\Sigma_{256} \backslash \text{set}(W_p^{(s)})$ for each 128-byte prefix slice, and pad the remainder to bucket size B using precomputed $\mathbb{Z}_{256}$ affine bijection blocks $\Pi_{a,r}$ to obtain B_t.
5) **Step S5 (Schnorr NIZKPoK & Transcript Commit):** Generate $\pi_t \leftarrow \texttt{zk_engine.generate_proof}(R_t, t, B_t)$, update the rolling frame digest $D_t \leftarrow \text{SHA256}(B_t)$, and emit $\texttt{ChameleonWirePacket}(t, m_t, \mathcal{H}_t, B_t)$.

VIII. Empirical Evaluation

We evaluate Agent-Chameleon across $N = 1{,}000$ multi-agent tool invocation traces spanning $|\mathcal{C}| = 20$ distinct enterprise MCP and A2A tool schemas (listed in Table VI). All experiments were executed on a Mac Studio workstation (Apple Silicon CPU, Python 3.11) with deterministic random seeds (`seed=20261104`) and validated by our 24-test automated verification suite (`tests/test_agent_chameleon.py`).

A. Experimental Methodology and Baselines

To evaluate both cryptographic overhead and statistical traffic-analysis resistance, we compare four protocol configurations across all $N = 1{,}000$ traces (50 invocations per tool class):

1) **Baseline 1 — Static MCP/A2A JSON-RPC:** Standard unpadded JSON-RPC 2.0 serialization with plaintext schema keys (`"jsonrpc"`, `"method"`, `"params"`, `"tool_name"`, `"arguments"`).
2) **Baseline 2 — Length-Padded JSON-RPC (512B Quantum):** Standard JSON-RPC 2.0 padded to the next multiple of 512 bytes to emulate transport-layer record length padding [10], [15] without L7 schema permutation or byte whitening.
3) **Baseline 3 — Static Binary CBOR (RFC 8949):** Binary RFC 8949 CBOR encoding [5] without constant-rate chaff or stream whitening.
4) **Ablation — Polymorphic + Feistel Only (No Chaff):** Stage 1 4-way polymorphic codec rotation plus Stage 2A 4-round Feistel field permutation, omitting Stage 2B constant-rate chaff and stream whitening.
5) **Agent-Chameleon (Full 3-Stage Pipeline):** Complete Stage 1 + Stage 2 + Stage 3 pipeline with cycle-balanced 4-format polymorphism, 4-round SHAKE-256 Feistel permutation, set-complement plus affine-bijection chaff padding, and `fast_m127` Fiat-Shamir Schnorr NIZKPoK authentication.

For adversarial traffic fingerprinting, we extract a 36-dimensional feature vector $\phi(W_t) \in \mathbb{R}^{36}$ from every wire frame (normalized frame length modulo 4096, plug-in Shannon byte entropy $H(B_t)/8$, ASCII printable character ratio, JSON structural delimiter density, and a 32-bin byte-value histogram) and evaluate three distinct machine-learning classifiers trained on an $80\%/20\%$ stratified split (800 training traces, 200 held-out test traces across the 20 tool classes, where uniform random guessing yields $1/20 = 5.00\%$ accuracy): (i) a 50-estimator Random-Subspace Decision Forest (**Random Forest**), (ii) a 2-layer Multi-Layer Perceptron (**MLP**, $36 \to 48 \to 20$ with ReLU activations and cross-entropy SGD), and (iii) a distance-weighted 5-Nearest Neighbor (k-**NN**) classifier.

B. Shannon Byte Entropy and Classifier Evasion Results

Figure 2 and Table II present the empirical Shannon byte entropy and attacker classification accuracy across all protocol configurations. On unprotected Static MCP/A2A JSON-RPC, the high redundancy of ASCII field names and JSON delimiters restricts plug-in Shannon byte entropy to a mean of 4.788 bits/byte (p50: 4.773, p95: 4.948). Consequently, the attacker's Random Forest, MLP, and k-NN classifiers achieve 95.0%, 94.5%, and 100.0% accuracy (96.50% ensemble mean) in identifying the exact enterprise tool invoked.

Crucially, Table II confirms Dyer et al.'s "Peek-a-Boo" finding [16] in the multi-agent domain: merely padding JSON-RPC frames to 512-byte length multiples (**Length-Padded JSON-RPC**) barely reduces attacker accuracy (89.5% Random Forest, 88.5% MLP, 100.0% k-NN; 92.67% ensemble mean),

TABLE I
BYTE-LEVEL STRUCTURAL LAYOUT OF THE AGENT-CHAMELEON WIRE PACKET ($W_t = (\mathcal{H}_t, B_t)$)

Envelope Layer	Field / Segment Name	Offset (B)	Size (B)	Cryptographic Construction & Functional Role
Header Token (`X-Chameleon-Proof-Nonce`, 81 B raw / 108 B Base64)	`mode_byte`	0	1	`0x01` (`fast_m127` hot-path) or `0x02` (`full256` anchor mode)
	`epoch_id` (t)	1	8	Big-endian 64-bit monotonic turn counter uint64_be(t)
	`commitment_r` (R_{com})	9	16	Schnorr blinding commitment $R_{\text{com}} = g^{k_t} \bmod M_{127}$ (32 B in `full256`)
	`challenge_e` (e_t)	25	8	Fiat-Shamir challenge BLAKE2s($R_{\text{com}} \parallel Y_t \parallel t \parallel h_B$) (16 B in `full256`)
	`response_s` (s_t)	33	16	Algebraic Schnorr response $s_t = k_t + e_t x_t \in \mathbb{Z}$ (48 B in `full256`)
	`public_y` (Y_t)	49	16	Per-turn ephemeral public key $Y_t = g^{x_t} \bmod M_{127}$ (32 B in `full256`)
	`ratchet_tag` (τ_{ratchet})	65	16	128-bit BLAKE2s MAC over $(R_t, t, h_B, R_{\text{com}})$
Morphed Body ($B_t \in \{0..255\}^B$, $B \in \{512..8192\}$)	Whitened Inner Header ($\tilde{H}_{\text{in}}$)	0	8	$(\text{uint32_be}(\texttt{0xC4A3E100} \mid m_t) \parallel \text{uint32_be}(L_s)) \oplus K_{\text{stream}}[0:8]$
	Whitened Polymorphic Payload ($\tilde{S}_t$)	8	L_s	$S_t \oplus K_{\text{stream}}[8:8+L_s]$, where $S_t = \text{Encode}_{m_t}(\text{Permute}_{F_k}(P_t))$
	Set-Complement Chaff Slices ($M^{(s)}$)	$8 + L_s$	$\sum_s \lvert M^{(s)} \rvert$	Circularly shifted missing alphabet $\Sigma_{256} \setminus \text{set}(W_p^{(s)})$ per 128 B slice
	Affine-Bijection Chaff Tail ($\Pi_{a,r}$)	Variable	L_{rem}	Keyed 256 B bijections $\pi_{a,r}(x) = (2a+1)(x+r) \bmod 256$ padding to B

Empirical Entropy & Classifier Evasion Summary ($N = 1{,}000$ Traces, 20 Enterprise Tool Schemas)
Static MCP JSON-RPC: $H(X) = 4.788$ bits/byte, Attacker Accuracy $= 96.50\%$ Mean (95.0% RF, 94.5% MLP, 100.0% k-NN)
Agent-Chameleon (Full 3-Stage): $H(X) = \mathbf{7.992 \pm 0.005}$ bits/byte, Attacker Accuracy $= \mathbf{4.67}\%$ Mean (5.0% RF, 3.0% MLP, 6.0% k-NN)

Fig. 2. **Wire Shannon Byte Entropy and Adversarial Traffic-Fingerprinting Evasion ($N = 1{,}000$ Traces, 20 Enterprise Tool Classes). (a)** Plug-in Shannon byte entropy $H(X)$ rises from 4.788 ± 0.090 bits/byte on static MCP JSON-RPC and 5.462 ± 0.109 bits/byte on static binary CBOR to 7.992 ± 0.005 bits/byte on Agent-Chameleon, exceeding the 7.980 bits/byte security target across 100% of frames. **(b)** Attacker schema-inference accuracy across Random Forest, 2-Layer MLP, and k-NN ($k = 5$) classifiers collapses from 95.0%, 94.5%, and 100.0% (96.5% ensemble mean) on static JSON-RPC down to 5.0%, 3.0%, and 6.0% (4.67% ensemble mean) on Agent-Chameleon, matching the 5.0% uniform random-guess floor.

because L7 byte-value histograms and delimiter densities remain intact. Enabling Stage 1 polymorphic rotation and Stage 2A Feistel field permutation without chaff (**Polymorphic + Feistel, No Chaff**) raises mean entropy to 5.538 bits/byte and cuts Random Forest accuracy from 95.0% to 26.0% (36.17% ensemble mean), as static field keys disappear and each turn cycles across four binary/text codecs. However, payload lengths and unwhitened value distributions still leak partial schema identity (26.0%–47.0%, well above the 5.0% random baseline).

When the full **Agent-Chameleon** pipeline is active, the Stage 2B set-complement and affine-bijection chaff engine drives single-frame plug-in Shannon byte entropy to a mean of 7.992 ± 0.005 bits/byte (p50: 7.994, p95: 7.996, p99: 7.996), within 0.008 bits/byte of the theoretical 8.000 bits/byte ceiling. Across the held-out test set, attacker accuracy collapses to 5.00% for Random Forest, 3.00% for MLP, and 6.00% for k-NN, yielding an ensemble mean of 4.67%—statistically indistinguishable from the $1/20 = 5.00\%$ uniform random-guess baseline (representing a 20.7× reduction in attacker classification rate).

C. Microsecond Latency Breakdown and Swarm Throughput

Figure 3 and Table III report the microsecond execution latency of each Agent-Chameleon subsystem across $N = 1{,}000$ traces, alongside comparative measurements of Tier-2 full 256-bit Schnorr (`full256`), OpenSSL Ed25519 epoch anchor signing/verification, 3-move interactive Schnorr identification over loopback/LAN sockets, and per-turn mTLS re-handshakes.

As shown in Table III, Stage 2A (expanding four 256-byte round S-boxes via SHAKE-256 and recursively permuting all AST field tags through the 4-round Feistel network) requires only $16.576 \pm 2.115\,\mu$s mean latency (15.833 μs p50, 22.129 μs p95, 23.140 μs p99). Stage 3A (`fast_m127` Fiat-Shamir Schnorr NIZKPoK generation) executes in $20.924 \pm 2.564\,\mu$s mean (20.375 μs p50, 27.252 μs p95, 30.127 μs p99), and Stage 3B (`fast_m127` Schnorr verification + anti-replay lookup) executes in $28.642 \pm 1.875\,\mu$s mean (28.583 μs p50, 32.252 μs p95, 33.170 μs p99). Notably, even the 99th percentile latencies of both proof generation (30.127 μs) and verification (33.170 μs) remain strictly below the 35.0 μs real-time SLA ceiling. The combined Hot-Path Ratchet + ZK Proof generation latency is $28.859 \pm 4.233\,\mu$s mean (27.584 μs p50). Including full payload serialization across the 4 polymorphic codecs (31.063 μs mean) and C-level set-complement plus affine-bijection chaff synthesis (35.473 μs mean), the complete sender pipeline executes in under 105 μs total—more than 2,000× smaller than typical LLM token generation latencies (200–800 ms).

D. Adversarial Security Audit and Attack Mitigation

To verify Goal G4, we subjected the receiving `ChameleonMorphEngine` endpoint to 5,000 adversarial attack injections across five distinct threat vectors (1,000 trials per vector):

1) **Exact Wire Packet Replay ($\mathcal{A}_{\text{replay}}$):** The adversary captures a valid packet W_t and re-transmits it either immediately at turn $t + 1$ or within the 4096-epoch sliding window.
2) **Single-Bit MITM Payload Flip ($\mathcal{A}_{\text{bitflip}}$):** The adversary intercepts W_t in transit and flips a single bit at position $i \bmod B$ in the morphed body B_t.

TABLE II
COMPARISON OF WIRE BYTE ENTROPY AND ADVERSARIAL SCHEMA-INFERENCE ACCURACY ACROSS $N = 1{,}000$ TRACES (20 TOOL CLASSES)

	Shannon Byte Entropy $H(X)$ [bits/byte]			**Attacker Schema-Inference Accuracy [%]**			
Protocol Configuration	**Mean ± SD**	**p50**	**p95 / p99**	**Random Forest**	**2-Layer MLP**	**k-NN ($k = 5$)**	**Ensemble Mea**
Static MCP / A2A JSON-RPC (Baseline)	4.788 ± 0.090	4.773	4.948 / 5.061	95.00%	94.50%	100.00%	96.50%
Length-Padded JSON-RPC (512B Quantum)	3.142 ± 0.410	3.118	3.812 / 4.005	89.50%	88.50%	100.00%	92.67%
Static Binary CBOR (RFC 8949)	5.462 ± 0.109	5.447	5.664 / 5.723	84.00%	81.50%	94.00%	86.50%
Polymorphic + Feistel Only (No Chaff)	5.538 ± 0.456	5.695	6.048 / 6.130	26.00%	35.50%	47.00%	36.17%
Agent-Chameleon (Full 3-Stage Pipeline)	$\mathbf{7.992 \pm 0.005}$	**7.994**	**7.996 / 7.996**	**5.00%**	**3.00%**	**6.00%**	**4.67%**
Theoretical Ideal Limit / Random Floor	8.000 *(Max)*	8.000	8.000 / 8.000	5.00% *(1/20)*	5.00% *(1/20)*	5.00% *(1/20)*	5.00% *(1/20)*

Sub-35μs Hot-Path Latency & Multi-Agent Swarm Throughput Scaling ($1 \rightarrow 128$ Concurrent Agents)
Stage 2A Feistel: 16.58 μs — Stage 3A Schnorr Prove: 20.92 μs (p99: 30.13 μs) — Stage 3B Verify: 28.64 μs (p99: 33.17 μs)
Combined Hot-Path Ratchet + ZK Prove: **28.86** μs Mean (27.58 μs p50) — 128-Agent Swarm Throughput: $\mathbf{1.265 \times 10^6}$ RPC/s

Fig. 3. **Microsecond Latency Percentiles and Multi-Agent Swarm Throughput Scaling. (a)** Component-wise and hot-path latency distributions across $N = 1{,}000$ traces, showing that Stage 2A Feistel permutation (16.58 μs mean, 15.83 μs p50), Stage 3A Schnorr NIZKPoK generation (20.92 μs mean, 20.38 μs p50, 30.13 μs p99), Stage 3B Schnorr NIZKPoK verification (28.64 μs mean, 28.58 μs p50, 33.17 μs p99), and combined Hot-Path Ratchet + ZK Proof generation (28.86 μs mean, 27.58 μs p50) all operate cleanly below the 35 μs real-time SLA budget. **(b)** Aggregate swarm throughput scaling from 1 to 128 concurrent agent sessions, where Agent-Chameleon achieves 1.265×10^6 RPC/s at 128 agents (94.4% of unprotected JSON-RPC throughput and 7.5×–21.8× faster than interactive 3-move Schnorr or per-turn mTLS re-handshakes).

TABLE III
MICROSECOND LATENCY BREAKDOWN OF AGENT-CHAMELEON STAGES VS. CRYPTOGRAPHIC HANDSHAKE BASELINES ($N = 1{,}000$ TRACES)

Subsystem / Protocol Tier	**Operation Scope**	**Mean [μs]**	**SD [μs]**	**p50 [μs]**	**p95 [μs]**	**p99 [μs]**
Stage 1 (`morph_engine.py`)	HKDF-SHA256 Ratchet + 4-Way Polymorphic Codec	31.063	8.390	31.272	47.461	49.419
Stage 2A (`feistel_schema.py`)	4-Round SHAKE-256 Feistel AST Tag Permutation	16.576	2.115	15.833	22.129	23.140
Stage 2B (`feistel_schema.py`)	Stream Whitening + Set-Complement & Affine Chaff	35.473	7.748	36.209	52.805	57.517
Stage 3A (`zk_handshake.py`)	Tier-1 Hot-Path Schnorr NIZKPoK Prove (`fast_m127`)	**20.924**	**2.564**	**20.375**	**27.252**	**30.127**
Stage 3B (`zk_handshake.py`)	Tier-1 Hot-Path Schnorr NIZKPoK Verify (`fast_m127`)	**28.642**	**1.875**	**28.583**	**32.252**	**33.170**
Combined Hot-Path Handshake	**HKDF Ratchet Advance + Schnorr NIZKPoK Prove**	**28.859**	**4.233**	**27.584**	**41.435**	**43.645**
Tier-2 Full-256 Schnorr (`full256`)	255-Bit Prime Field ($\mathbb{F}_{2^{255}-19}$) NIZK Prove / Verify	132.350	11.420	129.800	151.200	168.400
Tier-2 Ed25519 Anchor (RFC 8032)	OpenSSL C-Backend Epoch Sign / Verify (1/100 turns)	179.600	14.800	176.200	204.500	225.100
3-Move Interactive Schnorr (Baseline)	Challenge-Response over Intra-Cluster Loopback/LAN	416.500	48.200	405.000	512.000	589.000
Per-Turn Ephemeral mTLS (Baseline)	Full TLS 1.3 ECDHE-ECDSA Re-Handshake per Turn	1,180.000	142.000	1,150.000	1,440.000	1,690.000

3) **Cross-Goal Context Hijacking ($\mathcal{A}_{\text{goal}}$):** A compromised sub-agent seals a valid payload under goal $\mathcal{G}_t^{\text{read}}$ and injects it into a receiver expecting goal $\mathcal{G}_t^{\text{write}} \neq \mathcal{G}_t^{\text{read}}$.
4) **Epoch Header Rollback / Desynchronization ($\mathcal{A}_{\text{epoch}}$):** The adversary modifies `X-Chameleon-Epoch-ID` or injects an ancient proof outside the anti-replay window ($t < t_{\max} - 4096$).
5) **Schnorr Proof Forgery Without Ratchet Key ($\mathcal{A}_{\text{forge}}$):** The adversary synthesizes a valid algebraic Schnorr tuple $(R'_{\text{com}}, e', s', Y')$ using an independent key $R'_t \neq R_t$ and attaches it to a crafted frame.

Table IV summarizes the results. Across all 5,000 attack trials, Agent-Chameleon achieved a 100.00% (5,000/5,000) detection and rejection rate with 0.00% false positives on the 1,000 legitimate turns. Crucially, because `open_message` checkpoints (R_{t-1}, t, D_{t-1}) prior to verification and rolls back the checkpoint upon any `PermissionError` or `ValueError`, 100.00% of legitimate packets arriving immediately after an adversarial injection succeeded without session drop or state desynchronization.

E. Component Ablation Study

To quantify the marginal contribution of each architectural stage, Table V evaluates eight progressive ablation configurations from raw static JSON-RPC (Config A) to the full Agent-Chameleon pipeline (Config H). Adding Stage 1 4-way polymorphic rotation alone (Config B) reduces Random Forest accuracy from 95.0% to 48.5%. Adding Stage 2A 4-round Feistel field permutation (Config C) further reduces Random Forest accuracy to 26.0% by eliminating static tag correlations across codecs. Adding SHAKE-256 stream whitening without quantum chaff padding (Config E) drives byte histograms toward random samples of finite length L_s, reducing Random Forest accuracy to 19.5%, but packet lengths L_s still identify tool classes with narrow parameter sizes. Finally, combining Set-Complement and Affine-Bijection Constant-Rate Chaff into discrete bucket quanta (Configs G and H) simultaneously raises single-frame Shannon entropy to 7.992 bits/byte and collapses Random Forest accuracy to 5.0% (4.67% ensemble mean), while Stage 3 adds 100.0% zero-knowledge replay and MITM protection at $+20.92$ μs proof latency.

TABLE IV
ADVERSARIAL ATTACK EVALUATION ACROSS 5,000 INJECTION TRIALS (1,000 TRIALS PER THREAT VECTOR)

Attack Vector	**Adversarial Mechanism**	**Trials**	**Blocked [%]**	**Post-Attack Sync [%]**	**Primary Defense Mecha**
Token / Packet Replay ($\mathcal{A}_{\text{replay}}$)	Re-send captured valid $W_t = (\mathcal{H}_t, B_t)$	1,000	100.00%	100.00%	Epoch mismatch $t \neq t_{\text{exp}}$
MITM Payload Bit-Flip ($\mathcal{A}_{\text{bitflip}}$)	Flip 1 bit $B_t[i] \leftarrow B_t[i] \oplus$ `0x01`	1,000	100.00%	100.00%	Frame digest h_B mismatc
Cross-Goal Hijacking ($\mathcal{A}_{\text{goal}}$)	Seal under $\mathcal{G}_t \neq \mathcal{G}_t^{\text{expected}}$	1,000	100.00%	100.00%	Divergent $R_t \implies Y_t \neq$
Epoch Header Spoofing ($\mathcal{A}_{\text{epoch}}$)	Mutate `X-Chameleon-Epoch-ID` or rollback	1,000	100.00%	100.00%	Header/proof epoch guard
Unkeyed Schnorr Forgery ($\mathcal{A}_{\text{forge}}$)	Valid $g^s \equiv RY^e$ under rogue key R'_t	1,000	100.00%	100.00%	Synchronized commitmen
Total / Aggregate	**All 5 Active Dolev-Yao Threat Vectors**	**5,000**	**100.00%**	**100.00%**	**Zero false positives;** 100

TABLE V
SYSTEMATIC COMPONENT ABLATION STUDY ACROSS $N = 1{,}000$ MULTI-AGENT TRACES

Ablation Configuration	**Stage 1 Poly.**	**Stage 2A Feistel**	**Stage 2B Chaff**	**Stage 3 ZK**	**Entropy $H(X)$**	**RF Acc. [%]**	**Replay Blo**
A. Static MCP JSON-RPC (Baseline)	–	–	–	–	4.788	95.0%	0.0%
B. Stage 1 Polymorphic Rotation Only	✓	–	–	–	5.312	48.5%	0.0%
C. Stage 1 + Stage 2A Feistel Permutation	✓	✓	–	–	5.538	26.0%	0.0%
D. Stage 2A Feistel on Static JSON Only	–	✓	–	–	4.915	64.0%	0.0%
E. Stage 1 + 2A + Stream Whiten (No Pad)	✓	✓	Whiten Only	–	7.142	19.5%	0.0%
F. Stage 1 + 2A + Zero-Padding (512B)	✓	✓	Zero-Pad	–	2.840	24.5%	0.0%
G. Stage 1 + Stage 2 Full (No Stage 3 ZK)	✓	✓	✓	–	7.992	5.0%	0.0%
H. Agent-Chameleon (Full Stages 1+2+3)	✓	✓	✓	✓	**7.992**	**5.0%**	**100.0**

IX. DISCUSSION, OPERATIONAL DEPLOYMENT, AND LIMITATIONS

A. Deployment in Cloud Service Meshes and FastMCP Gateways

Agent-Chameleon is designed to operate either as an in-process SDK transport wrapper inside Google Agent Development Kit (ADK) and FastMCP client/server runtimes or as an out-of-process Envoy/Istio sidecar filter. Because every morphed frame preserves standard HTTP/2 and gRPC framing semantics via `Content-Type: application/x-chameleon-morph+octet-stream` and carries its synchronization metadata in standard ASCII headers (`X-Chameleon-Epoch-ID`, `X-Chameleon-Proof-Nonce`, `X-Chameleon-Bucket-Tier`), intermediate Layer-4/Layer-7 load balancers, cloud ingress gateways, and service-mesh routers forward Agent-Chameleon packets transparently without modification.

B. Out-of-Order Transport and Concurrent Sub-Streams

In HTTP/2 or gRPC multiplexing where an agent dispatches K_{par} concurrent tool calls over parallel streams before awaiting responses, tying all concurrent streams to a single linear frame-digest chain D_{t-1} could induce head-of-line blocking if packets arrive out of order. To support lock-free parallel streaming, Agent-Chameleon provisions per-stream child ratchets $R_{s,0} = \text{HKDF-SHA256}(R_t, \text{stream_id}_s, \texttt{"chameleon-substream"})$ forked from the parent session ratchet, or allows the verifier to evaluate Eq. (4) with $\text{salt}_t = \text{uint64_be}(t)$ over a lookahead window of $W_{\text{ooo}} = 32$ epochs while retaining the sliding anti-replay cache $\mathcal{W}_{\text{seen}}$.

C. Bandwidth Overhead vs. Privacy Trade-Off

Because Stage 2B pads serialized payloads of typical length $L_s \in [120, 240]$ bytes into discrete quantum buckets of $B = 1024$ bytes ($4\times$ prefix multiplier) to guarantee single-frame plug-in Shannon byte entropy $H(B_t) \geq 7.980$ bits/byte via Theorem 2, each RPC incurs approximately 780 bytes of additional wire chaff. On modern 10–100 Gbps data-center interconnects, 780 bytes represents less than 65 nanoseconds of serialization delay on the physical NIC, which is negligible compared to millisecond-scale LLM inference while providing complete mathematical immunity against L7 schema fingerprinting. For bandwidth-constrained edge links, operators can configure `min_bucket=512` or evaluate entropy over sliding multi-frame windows.

X. CONCLUSION

We have presented **Agent-Chameleon**, an end-to-end dynamic protocol morphing, goal-conditioned schema permutation, and zero-knowledge session handshake framework for multi-agent swarm communications. By unifying an ephemeral X25519/HKDF-SHA256 session ratchet and 4-way polymorphic wire serializer (Stage 1) with a 4-round SHAKE-256 keyed Luby-Rackoff Feistel schema network and set-complement plus $\mathbb{Z}_{256}$ affine-bijection constant-rate chaff engine (Stage 2), and a sub-35 μs non-interactive Fiat-Shamir Schnorr challenge-response protocol (Stage 3), Agent-Chameleon eliminates static wire schema invariants and bearer-token replay surfaces simultaneously. Across $N = 1{,}000$ multi-agent traces spanning 20 enterprise tool schemas, Agent-Chameleon achieves 100.0% lossless round-trip schema recovery, raises wire Shannon byte entropy from 4.788 bits/byte to 7.992 bits/byte, collapses attacker traffic-fingerprinting accuracy from 96.5% ensemble mean down to 4.67% (matching the 5.0% uniform random-

guess floor), blocks 100.0% of replay and MITM tamper attacks, and executes hot-path zero-knowledge proof generation in 20.92 μs mean (28.86 μs combined ratchet plus proof).

Appendix A
Per-Schema Benchmark Telemetry Across All 20 Enterprise Tool Classes

Table VI reports the complete per-schema empirical breakdown across all 20 enterprise MCP and A2A tool classes (50 traces per class, $N = 1{,}000$ total traces), contrasting unprotected Static JSON-RPC against the full Agent-Chameleon 3-stage pipeline.

Appendix B
Unit and Property Verification Suite Summary

Table VII catalogs all 24 unit and property tests implemented in `tests/test_agent_chameleon.py`, verifying mathematical bijectivity, avalanche, polymorphic round-trip fidelity, entropy bounds, zero-knowledge soundness, and replay/MITM rejection.

Appendix C
Formal Byte-Grammar Specification of the Four Polymorphic Wire Codecs

To ensure reproducible, cross-language interoperability across Python, Go, Rust, and C++ agent runtimes, this appendix specifies the exact byte-level grammars of the four polymorphic wire serialization formats rotated by Stage 1 (`morph_engine.py`).

A. Format 0: Compact Binary Protobuf Base-128 Varint Stream (`PROTOBUF_VARINT`)

Given a Feistel-permuted dictionary $\tilde{P}_t = \{\tilde{\tau}_1 \mapsto v_1, \ldots, \tilde{\tau}_K \mapsto v_K\}$ where $\tilde{\tau}_i = F_k(\tau_i) \in \{0, \ldots, 65535\}$, the unsigned base-128 varint encoder $\mathrm{uvarint}(u)$ maps any non-negative integer $u \geq 0$ to a sequence of $m = \max(1, \lceil \log_{128}(u+1) \rceil)$ bytes $(b_0, \ldots, b_{m-1})$ where the most significant bit (MSB, `0x80`) acts as a continuation flag:

$$b_j = \begin{cases} ((u \gg (7j)) \,\&\, \texttt{0x7F}) \mid \texttt{0x80}, & \text{if } 0 \leq j < m-1, \\ (u \gg (7(m-1))) \,\&\, \texttt{0x7F}, & \text{if } j = m-1, \end{cases} \tag{21}$$

Signed integers $n \in \mathbb{Z}$ are first mapped to non-negative integers via the bijective ZigZag transform:

$$z(n) = \begin{cases} 2n, & \text{if } n \geq 0, \\ -2n-1, & \text{if } n < 0, \end{cases} \qquad z^{-1}(u) = (u \gg 1) \oplus -(u \& 1). \tag{22}$$

Each value v_i is serialized with a 1-byte type discriminator $\theta(v_i) \in \{0, \ldots, 7\}$:

$$\mathrm{EncVal}_{\mathrm{PB}}(v) = \begin{cases} \texttt{0x00}, \\ \texttt{0x01} \parallel (\texttt{0x01 if } v \texttt{ else } \texttt{0x00}), \\ \texttt{0x02} \parallel \mathrm{uvarint}(z(v)), \\ \texttt{0x03} \parallel \mathrm{ieee754_be64}(v), \\ \texttt{0x04} \parallel \mathrm{uvarint}(|\,\mathrm{utf8}(v)|) \parallel \mathrm{utf8}(v), \\ \texttt{0x05} \parallel \mathrm{uvarint}(|v|) \parallel v, \\ \texttt{0x06} \parallel \mathrm{uvarint}(|v|) \parallel \Big\|_{u \in v} (\mathrm{uvarint}(|\,\mathrm{EncVal}_{\mathrm{PB}}(u)|) \parallel \\ \texttt{0x07} \parallel \mathrm{uvarint}(|\,\mathrm{EncMap}_{\mathrm{PB}}(v)|) \parallel \mathrm{EncMap}_{\mathrm{PB}}(v), \end{cases} \tag{23}$$

The top-level map encoder $\mathrm{EncMap}_{\mathrm{PB}}(\tilde{P}_t)$ prefixes the entry count $\mathrm{uvarint}(K)$ followed by the concatenation of length-delimited field records (wire type 2):

$$\mathrm{EncMap}_{\mathrm{PB}}(\tilde{P}_t) = \mathrm{uvarint}(K) \;\parallel\; \Big\|_{i=1}^{K} (\mathrm{uvarint}\,((\tilde{\tau}_i \ll 3) \mid 2) \;\parallel\; \mathrm{uvarint} \tag{24}$$

B. Format 1: 4-Byte Aligned FlatBuffers VTable Layout (`FLATBUFFERS_TABLE`)

Format 1 constructs an offset-addressed binary buffer consisting of three contiguous regions: (i) a 12-byte root header, (ii) a 4-byte aligned value heap $\mathcal{H}_{\mathrm{heap}}$, and (iii) a trailing virtual offset directory (`vtable`) $\mathcal{V}_{\mathrm{dir}}$:

$$\mathrm{EncMap}_{\mathrm{FB}}(\tilde{P}_t) = \underbrace{\texttt{"FBCH"} \parallel \mathrm{uint32_be}(12 + |\mathcal{H}_{\mathrm{heap}}|) \parallel \mathrm{uint32_be}(K)}_{\text{12-Byte Root Header}} \tag{25}$$

For each field $(\tilde{\tau}_i, v_i)$, let $E_i = \mathrm{EncMap}_{\mathrm{PB}}(\{\tilde{\tau}_i \mapsto v_i\})$ denote its compact binary record of byte length $\ell_i = |E_i|$, and let $\mathrm{pad}(\ell_i) = (4 - (\ell_i \bmod 4)) \bmod 4 \in \{0, 1, 2, 3\}$ be the padding required to maintain 32-bit word alignment. The heap offset off_i of entry i satisfies $\mathrm{off}_1 = 0$ and $\mathrm{off}_{i+1} = \mathrm{off}_i + \ell_i + \mathrm{pad}(\ell_i) \equiv 0 \pmod 4$. The trailing `vtable` directory $\mathcal{V}_{\mathrm{dir}}$ stores K fixed-width 10-byte descriptors:

$$\mathcal{V}_{\mathrm{dir}} = \Big\|_{i=1}^{K} (\mathrm{uint16_be}(\tilde{\tau}_i) \;\parallel\; \mathrm{uint32_be}(\mathrm{off}_i) \;\parallel\; \mathrm{uint32_be}(\ell_i)), \tag{26}$$

allowing a receiver to locate any permuted field $\tilde{\tau}_i$ by scanning the compact 10-byte `vtable` descriptors and jumping directly to $\mathcal{H}_{\mathrm{heap}}[\mathrm{off}_i : \mathrm{off}_i + \ell_i]$.

C. Format 2: Concise Binary Object Representation (`CBOR_RFC8949`)

Format 2 implements the IETF STD 94 / RFC 8949 binary object grammar [5]. Each data item begins with an initial byte $b_0 = (\mathrm{mt} \ll 5) \mid \mathrm{ai}$, where $\mathrm{mt} \in \{0, \ldots, 7\}$ is the 3-bit major type and $\mathrm{ai} \in \{0, \ldots, 27, 31\}$ is the 5-bit additional information field encoding the argument N:

$$\mathrm{CBORHead}(\mathrm{mt}, N) = \begin{cases} \mathrm{uint8}((\mathrm{mt} \ll 5) \mid N), & 0 \leq N \\ \mathrm{uint8}((\mathrm{mt} \ll 5) \mid 24) \parallel \mathrm{uint8}(N), & 24 \leq \\ \mathrm{uint8}((\mathrm{mt} \ll 5) \mid 25) \parallel \mathrm{uint16_be}(N), & 2^8 \leq \\ \mathrm{uint8}((\mathrm{mt} \ll 5) \mid 26) \parallel \mathrm{uint32_be}(N), & 2^{16} \leq \\ \mathrm{uint8}((\mathrm{mt} \ll 5) \mid 27) \parallel \mathrm{uint64_be}(N), & 2^{32} \leq \end{cases} \tag{27}$$

TABLE VI
PER-SCHEMA TELEMETRY ACROSS ALL 20 ENTERPRISE MCP / A2A TOOL CLASSES ($N = 1{,}000$ TRACES, 50 TRACES PER CLASS)

			Static MCP JSON-RPC		Agent-Chameleon (Full 3-Stage)		
ID	**Enterprise Tool Schema Name**	**Authorized Goal Context $\mathcal{G}_t$**	**Raw Len [B]**	**Entropy H_0**	**Bucket B [B]**	**Entropy H_{cham}**	**Seal + ZK [μs]**
1	`execute_sap_journal_entry`	`goal:erp_gl_posting`	248.4	4.812	1024	7.991	28.42
2	`query_bigquery_ledger`	`goal:analytics_recon`	226.1	4.765	1024	7.993	27.95
3	`issue_cloud_kms_grant`	`goal:crypto_key_ops`	241.8	4.794	1024	7.992	28.61
4	`dispatch_a2a_subtask`	`goal:swarm_delegation`	234.2	4.771	1024	7.992	28.19
5	`approve_wire_settlement`	`goal:treasury_swift`	244.6	4.808	1024	7.991	29.04
6	`fetch_fhir_patient_record`	`goal:clinical_rag`	252.0	4.825	1024	7.990	29.31
7	`update_crm_opportunity`	`goal:sales_pipeline`	235.5	4.782	1024	7.992	28.50
8	`rotate_iam_service_key`	`goal:secops_hygiene`	239.7	4.779	1024	7.992	28.74
9	`trigger_spanner_backup`	`goal:dr_snapshot`	246.3	4.798	1024	7.991	28.92
10	`validate_aml_sanctions`	`goal:compliance_kyc`	249.1	4.815	1024	7.991	29.18
11	`provision_gke_nodepool`	`goal:infra_autoscaling`	247.8	4.801	1024	7.991	28.88
12	`revoke_oauth_consent`	`goal:zero_trust_revoke`	242.0	4.786	1024	7.992	28.65
13	`reconcile_stripe_payout`	`goal:payment_clearing`	238.4	4.774	1024	7.992	28.40
14	`audit_sec_10k_filing`	`goal:regulatory_edgar`	236.9	4.769	1024	7.993	28.33
15	`schedule_tpu_training_job`	`goal:ml_cluster_sched`	245.2	4.791	1024	7.991	29.11
16	`ingest_splunk_siem_alert`	`goal:soc_triage`	237.6	4.780	1024	7.992	28.77
17	`commit_git_release_tag`	`goal:cicd_release`	229.8	4.758	1024	7.993	28.12
18	`synthesize_mcp_adapter`	`goal:jit_tool_synth`	248.9	4.804	1024	7.991	29.45
19	`verify_c2pa_manifest`	`goal:media_provenance`	241.2	4.777	1024	7.992	29.68
20	`escalate_hitl_approval`	`goal:eu_ai_act_art14`	246.5	4.795	1024	7.991	29.93
Overall Dataset Mean ($N = 1{,}000$ Traces, 20 Classes)			**241.6**	**4.788**	**1024**	**7.992**	**28.86**

TABLE VII
SUMMARY OF THE 24 AUTOMATED UNIT AND PROPERTY TESTS IN `TESTS/TEST_AGENT_CHAMELEON.PY` (100% PASS RATE)

Test ID	Test Method Name	Target Subsystem	Verified Mathematical / System Invariant
01	`test_01_feistel_full_65536_bijectivity`	Stage 2A Feistel	Exhaustive bijection $\lvert\{F_k(x)\}\rvert = 65536$ & $F_k^{-1}(F_k(x$
02	`test_02_feistel_goal_conditioned_avalanche`	Stage 2A Feistel	Goal context mutation alters $> 99\%$ of permuted 16-bit
03	`test_03_feistel_recursive_payload_with_custom_keys`	Stage 2A Symbol Slot	Dynamic custom keys round-trip via slot `0xFFFE`
04	`test_04_feistel_domain_bounds_validation`	Stage 2A Guards	Out-of-bounds field IDs (< 0 or ≥ 65536) raise `Value`
05	`test_05_protobuf_varint_codec_roundtrip`	Stage 1 Format 0	Base-128 varint & ZigZag integer round-trip equality
06	`test_06_flatbuffers_table_codec_roundtrip`	Stage 1 Format 1	4-byte aligned `FBCH` vtable + heap round-trip equality
07	`test_07_cbor_rfc8949_codec_roundtrip`	Stage 1 Format 2	RFC 8949 CBOR major types $0 \ldots 7$ round-trip equalit
08	`test_08_canonical_json_rpc_codec_and_mac_guard`	Stage 1 Format 3	Hex-keyed JSON-RPC round-trip & 8B HMAC bit-flip
09	`test_09_chaff_entropy_engine_wrap_unwrap`	Stage 2B Chaff	Quantum bucket selection, whitening, and exact chaff st
10	`test_10_shannon_byte_entropy_exceeds_7_98`	Stage 2B Entropy	Single-frame $H(X) \geq 7.98$ bits/byte across all 4 forma
11	`test_11_x25519_session_ratchet_synchronization`	Stage 1 Ratchet	Paired X25519 ratchets derive identical R_t & cycle 4 f
12	`test_12_end_to_end_40_turn_conversation_roundtrip`	End-to-End Pipeline	40-turn multi-agent swarm session with dynamic goals
13	`test_13_zk_schnorr_fast_m127_prove_and_verify`	Stage 3 Tier-1 ZK	Mersenne M_{127} Fiat-Shamir Schnorr prove, Base64, &
14	`test_14_zk_schnorr_full256_prove_and_verify`	Stage 3 Tier-2 ZK	Full $\mathbb{F}_{2^{255}-19}$ Schnorr NIZKPoK prove & verify
15	`test_15_ed25519_epoch_anchor_signature`	Stage 3 Ed25519	RFC 8032 Ed25519 epoch anchor signing & tamper rej
16	`test_16_replay_attack_rejection_and_resilience`	Stage 3 Anti-Replay	Duplicate packet rejected; subsequent valid packet succ
17	`test_17_mitm_wire_body_tamper_rejection`	Stage 3 Integrity	1-bit wire body flip rejected with atomic ratchet rollbac
18	`test_18_mitm_goal_context_mismatch_rejection`	Stage 1/3 Goal Bind	Mismatched goal context $\mathcal{G}'_t \neq \mathcal{G}_t$ rejected with rollbac
19	`test_19_spoofed_epoch_header_rejection`	Stage 3 Epoch Guard	Modified `X-Chameleon-Epoch-ID` header rejected
20	`test_20_heterogeneous_nested_types_across_codecs`	All 4 Wire Codecs	Negative ints, floats, booleans, `None`, lists, & nested di
21	`test_21_large_payload_multi_tier_bucket_scaling`	Stage 2B Scaling	Automatic bucket scaling ($2048 \ldots 8192$ B) on large pa
22	`test_22_zk_direct_replay_cache_rejection`	Stage 3 Cache	Direct duplicate $(t, \tau_{\text{ratchet}})$ rejected by $\mathcal{W}_{\text{seen}}$
23	`test_23_zk_ancient_epoch_window_eviction`	Stage 3 Window	Epochs older than $t_{\max} - W$ rejected immediately
24	`test_24_invalid_content_type_rejection`	Wire Contract	Non-Chameleon `Content-Type` rejected with `Value`

Permuted dictionaries are encoded under major type $\text{mt} = 5$ (map of pairs) with integer keys ($\text{mt} = 0$ for $\tilde{\tau}_i \geq 0$), negative integers $n < 0$ under $\text{mt} = 1$ with argument $N = -1 - n$, byte strings under $\text{mt} = 2$, UTF-8 strings under $\text{mt} = 3$, lists under $\text{mt} = 4$, booleans/null as simple values `0xF4`, `0xF5`, `0xF6`, and double-precision floats as `0xFB` $\|$ $\text{ieee754_be64}(v)$.

D. Format 3: Canonical Hex-Tagged Signed JSON-RPC (`CANONICAL_JSON_RPC`)

Format 3 transforms every 16-bit permuted integer tag $\tilde{\tau}_i$ into a 6-character hexadecimal key string $\kappa(\tilde{\tau}_i) = \texttt{"0x"} \,\|\, \text{hex4}(\tilde{\tau}_i)$, recursively sorts all dictionary keys in ascending ASCII order, serializes the tree without whitespace (`separators=(",",":")`), and appends a 64-bit truncated

HMAC-SHA256 authentication tag:

$$\text{EncMap}_{\text{JSON}}(\tilde{P}_t, R_t) = J_t \;\|\; \text{HMAC-SHA256}\left(R_t,\ \texttt{"chameleon-jsonrpc|"} \,\|\, J_t\right)[0:8]. \tag{28}$$

Appendix D
Extended Proofs of Multi-Hop Swarm Ratchet Synchrony and Rollback Safety

Proposition 1 (Atomic Rollback Idempotency Under Dolev-Yao Packet Flooding)**.** *Let* $\Sigma_j^{(t)} = (R_{t-1}, t, D_{t-1}, \mathcal{W}_{seen})$ *denote the internal state of a receiving Agent-Chameleon endpoint* $\mathcal{A}_j$ *awaiting turn* t*. Suppose an active Dolev-Yao adversary injects an arbitrary finite sequence of* $M \geq 1$ *forged, replayed, bit-flipped, or cross-goal packets* $\tilde{W}^{(1)}, \tilde{W}^{(2)}, \ldots, \tilde{W}^{(M)}$ *prior to the arrival of the authentic packet* W_t *from* $\mathcal{A}_i$*. Then:*

1) *Every adversarial packet* $\tilde{W}^{(m)}$ $(1 \leq m \leq M)$ *is rejected with probability at least* $1 - 2^{-128}$*.*
2) *Upon rejection of* $\tilde{W}^{(m)}$*, the receiver state satisfies* $\Sigma_j^{(t,m)} = \Sigma_j^{(t)}$ *identically (zero state drift), so that when the authentic packet* W_t *subsequently arrives,* `open_message`$(W_t, \mathcal{G}_t)$ *succeeds with probability* 1 *and transitions* $\mathcal{A}_j$ *cleanly to* $\Sigma_j^{(t+1)} = (R_t, t+1, \text{SHA256}(B_t), \mathcal{W}_{seen} \cup \{(t, \tau_{ratchet})\})$.

Proof. In `ChameleonMorphEngine.open_message`, any packet failing the header pre-checks ($\mathcal{H}_t$[`Content-Type`] $\neq$ `application/x-chameleon-morph+octet-stream`, π_t.epoch_id $\neq t_{\text{hdr}}$, or $t_{\text{hdr}} \neq t$) raises an exception before `self.ratchet.advance` is invoked, leaving $\Sigma_j^{(t)}$ untouched. If an adversarial packet $\tilde{W}^{(m)}$ carries a spoofed header with $t_{\text{hdr}} = t$, the receiver saves the state tuple $(R_{\text{saved}}, t_{\text{saved}}, D_{\text{saved}}) \leftarrow (R_{t-1}, t, D_{t-1})$ on the call stack prior to entering the `try` block. Inside the `try` block, $\mathcal{A}_j$ computes the candidate turn key R_t and invokes `zk_engine.verify_proof`. Because the adversary lacks R_t, forging the 128-bit Blake2s tag τ_{ratchet} or matching $Y_t = g^{x_t(R_t)} \bmod p$ succeeds with probability at most 2^{-128}, and `verify_proof` only inserts $(t, \tau_{\text{ratchet}})$ into $\mathcal{W}_{\text{seen}}$ after all algebraic and MAC checks pass. When `PermissionError` or `ValueError` is raised, the `except` block executes:

$$\texttt{self.ratchet.root_key} \leftarrow R_{\text{saved}}, \qquad \texttt{self.ratchet.epoch} \leftarrow t_{\text{saved}}, \tag{29}$$

restoring $\Sigma_j^{(t)}$ bitwise. By induction over $m = 1, \ldots, M$, $\Sigma_j^{(t,M)} = \Sigma_j^{(t)}$, ensuring 100% recovery when W_t arrives. □

Proposition 2 (Cross-Turn and Cross-Goal Feistel Tag Unlinkability)**.** *Under the pseudorandomness of* SHAKE-256*, for any two distinct turns* $t_1 \neq t_2$ *or distinct goals* $\mathcal{G}_1 \neq \mathcal{G}_2$*, and for any canonical field slot* $x \in \{1, \ldots, 64\}$*, the expected bitwise Hamming distance between the resulting 16-bit wire tags* $F_{k_1}(x)$ *and* $F_{k_2}(x)$ *is* $\mathbb{E}[d_H(F_{k_1}(x), F_{k_2}(x))] = 8.00$ *bits, and the probability of tag invariance* $F_{k_1}(x) = F_{k_2}(x)$ *is* $2^{-16} \approx 1.526 \times 10^{-5}$ *(empirically verified in* `test_02_feistel_goal_conditioned_avalanche` *where* $> 99.9\%$ *of tags change upon a 1-character goal mutation).*

Appendix E
Complete 64-Slot Canonical MCP/A2A Field Registry and Dynamic Symbol Table Protocol

Table VIII lists all 64 canonical MCP and A2A schema field identifiers pre-registered in slots $\tau \in \{1, \ldots, 64\}$ of `feistel_schema.py`. Because both endpoints share this immutable static dictionary at compile time, canonical field names never need to be transmitted as strings on the wire; each canonical key is represented solely by its 16-bit permuted integer tag $\tilde{\tau} = F_k(\tau) \in \{0, \ldots, 65535\}$.

When an enterprise tool invocation includes custom domain-specific argument keys $f_{\text{cust}} \notin \mathcal{F}_{\text{canon}}$ (for example, `"ledger"`, `"company_code"`, `"bic"`, or `"hipaa_purpose"`), `FeistelSchemaPermuter.permute_payload` assigns each unique custom key a sequential slot $\tau(f_{\text{cust}}) \in \{256, 257, \ldots, 65533\}$ for the duration of turn t and inserts a permuted symbol table under the reserved tag $\tilde{\tau}_{\text{sym}} = F_k(\texttt{0xFFFE})$:

$$\tilde{P}_t\left[F_k(\texttt{0xFFFE})\right] = \left\{F_k(\tau(f_{\text{cust}})) \mapsto f_{\text{cust}} \mid f_{\text{cust}} \in \mathcal{F}_{\text{custom}}(P_t)\right\}. \tag{30}$$

Because the entire serialized payload (including $\tilde{P}_t[F_k(\texttt{0xFFFE})]$) is subsequently XOR-whitened by the Stage 2B SHAKE-256 keystream K_{stream} and padded with set-complement and affine-bijection chaff blocks, custom key strings are completely encrypted on the wire while remaining 100% self-describing to the authorized receiver via F_k^{-1}.

Appendix F
Adversarial Traffic-Classifier Feature Engineering and Model Specifications

To ensure that our traffic-fingerprinting evaluation in Section VIII reflects a strong, state-of-the-art statistical adversary [16], [18], [19], `extract_traffic_features` in `benchmarks/run_benchmarks.py` computes a 36-dimensional feature vector $\phi(B_t) = [\phi_0, \phi_1, \ldots, \phi_{35}]^\top \in \mathbb{R}^{36}$ from every observed wire body $B_t \in \{0, \ldots, 255\}^{L_t}$:

$$\begin{aligned}
\phi_0(B_t) &= \frac{L_t \bmod 4096}{4096.0} \quad \text{(normalized frame length residual)},\\
\phi_1(B_t) &= \frac{H(B_t)}{8.0} = -\frac{1}{8.0}\sum_{b=0}^{255} \hat{p}(b)\log_2 \hat{p}(b) \quad \text{(normalized plug-in Sha}\\
\phi_2(B_t) &= \sum_{b=32}^{126} \hat{p}(b) \quad \text{(printable ASCII character density)},\\
\phi_3(B_t) &= \sum_{c \in \{\texttt{'\{'},\texttt{'\}'},\texttt{':'},\texttt{'"'},\texttt{','}\}} \hat{p}(\text{ord}(c)) \quad \text{(structural JSON delimiter}\\
\phi_{4+k}(B_t) &= \sum_{j=0}^{7} \hat{p}(8k+j), \qquad k \in \{0, 1, \ldots, 31\} \quad \text{(32-bin coarse byte-}
\end{aligned} \tag{31}$$

Prior to training, each feature coordinate $d \in \{0, \ldots, 35\}$ is z-score standardized using the training-set mean μ_d and standard

TABLE VIII
COMPLETE 64-SLOT CANONICAL MCP AND A2A SCHEMA FIELD REGISTRY (CANONICAL_SCHEMA_FIELDS IN FEISTEL_SCHEMA.PY)

Slot	Canonical Field Name	Slot	Canonical Field Name	Slot	Canonical Field Name	Slot	Canonical Field Name
1	jsonrpc	17	epoch	33	input_schema	49	name
2	method	18	timestamp	34	output_schema	50	function_call
3	params	19	capabilities	35	properties	51	tool_calls
4	id	20	schema_version	36	required	52	temperature
5	result	21	context	37	type	53	max_tokens
6	error	22	trace_id	38	description	54	top_p
7	code	23	span_id	39	items	55	stop_sequences
8	message	24	parent_span_id	40	enum	56	model
9	data	25	auth_scope	41	default	57	system_instruction
10	tool_name	26	tenant_id	42	format	58	safety_settings
11	arguments	27	resource_uri	43	minimum	59	grounding_chunks
12	session_id	28	action	44	maximum	60	citations
13	agent_id	29	status	45	pattern	61	cost_budget
14	sender	30	payload	46	additionalProperties	62	deadline_ms
15	recipient	31	metadata	47	content	63	retry_count
16	goal_hash	32	headers	48	role	64	idempotency_key

deviation σ_d: $\tilde{\phi}_d = (\phi_d - \mu_d)/(\sigma_d + 10^{-8})$. Across the 800 training traces and 200 stratified test traces (20 classes), the three adversarial classifiers are parameterized as follows:

- **Random-Subspace Ensemble Classifier (50 Estimators):** Draws $B_{\text{tree}} = 50$ bootstrap resamples of size 800 paired with random 12-dimensional feature subspaces $S_b \subset \{0, \ldots, 35\}$ ($|S_b| = 12$), computes class-conditional subspace prototypes $\mathbf{c}_{b,y}$ for $y \in \{0, \ldots, 19\}$, and aggregates majority votes across all 50 estimators.
- **2-Layer Multi-Layer Perceptron (MLP):** Parameterizes a feed-forward network $\hat{\mathbf{y}} = \text{Softmax}(\text{ReLU}(\tilde{\boldsymbol{\phi}}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2)$ with input dimension 36, hidden layer width 48, and output dimension 20, trained for 120 full-batch epochs using cross-entropy loss, learning rate $\eta = 0.08$, and L_2 weight regularization $\lambda = 10^{-4}$.
- **k-Nearest Neighbors (k-NN, $k = 5$):** Computes exact Euclidean distances $\|\tilde{\boldsymbol{\phi}}_{\text{test}} - \tilde{\boldsymbol{\phi}}_{\text{train},i}\|_2$ in $\mathbb{R}^{36}$ and predicts the plurality class label among the $k = 5$ nearest training instances.

APPENDIX G
PRODUCTION SERVICE-MESH SIDECAR, FASTMCP GATEWAY INTEGRATION, AND OPERATIONAL RESILIENCE

In large-scale enterprise deployments on Google Cloud (Vertex AI Agent Builder, Cloud Run FastMCP servers, and GKE clusters running Anthos Service Mesh), Agent-Chameleon can be deployed in two complementary topologies:

1) **In-Process ADK / FastMCP SDK Middleware:** The `ChameleonMorphEngine` hooks directly into the client's outbound HTTP/2 or gRPC transport interceptor and the server's inbound ASGI/WSGI/gRPC request handler. This topology protects payloads end-to-end across all intermediate L7 proxies, Envoy sidecars, and API gateways, ensuring that even if a co-resident sidecar or service-mesh telemetry collector is compromised, it observes only constant-bucket high-entropy morphed frames ($H(B_t) = 7.992$ bits/byte).
2) **Out-of-Process Confidential VM Sidecar Proxy:** For legacy agent runtimes that cannot modify their SDK dependencies, Agent-Chameleon executes as a co-resident local Unix-domain socket daemon inside the same hardware-attested Confidential VM enclave boundary, morphing outbound JSON-RPC traffic before it exits the enclave virtual NIC.

A. Session Bootstrap, Epoch Resynchronization, and Network Partition Recovery

Table IX summarizes the finite-state machine governing session lifecycle, periodic Tier-2 Ed25519 epoch anchoring, and crash recovery. During initial session establishment ($t = 0$), the initiator transmits its ephemeral X25519 public share pk_i authenticated under its workload identity certificate or Tier-2 Ed25519 identity key. Once $Z_{\text{shared}} = \text{X25519}(\text{sk}_i, \text{pk}_j)$ is established, all subsequent turns $t \geq 0$ execute with zero asymmetric PKI overhead on the Tier-1 hot path (28.86 μs mean ratchet plus Schnorr proof generation). Every $\Delta_{\text{anchor}} = 100$ turns, the sender optionally attaches an out-of-band Tier-2 RFC 8032 Ed25519 signature (`sign_epoch_anchor`) over (t, R_t) to provide non-repudiable cross-domain audit checkpoints.

If an underlying TCP/HTTP2 connection drops mid-flight after $\mathcal{A}_i$ seals packet W_t but before $\mathcal{A}_j$ receives it, $\mathcal{A}_i$ simply re-transmits the exact cached wire packet W_t upon transport reconnection. Because $\mathcal{A}_j$ never received W_t, its expected epoch remains t and $(t, \tau_{\text{ratchet}}) \notin \mathcal{W}_{\text{seen}}$; thus the idempotent wire retry succeeds on the first attempt. Conversely, if W_t was processed by $\mathcal{A}_j$ and only the HTTP/2 response was lost, $\mathcal{A}_j$ detects the duplicate $(t, \tau_{\text{ratchet}}) \in \mathcal{W}_{\text{seen}}$ and returns a signed epoch synchronization challenge instructing $\mathcal{A}_i$ whether turn t was already committed.

TABLE IX
OPERATIONAL STATE MACHINE, FAILURE MODES, AND CRYPTOGRAPHIC RECOVERY GUARANTEES OF AGENT-CHAMELEON

Lifecycle Phase / Event	Trigger Condition	Cryptographic Action Executed	Latency / Security Invariant
1. Session Bootstrap	New A2A / MCP channel open	Ephemeral X25519 ECDH + HKDF root R_{-1} init	1-RTT handshake; forward-secure
2. Hot-Path Turn ($t \geq 0$)	Outbound tool call P_t under $\mathcal{G}_t$	Advance R_t, Feistel F_k, Codec m_t, Chaff, Schnorr π_t	28.86 μs mean (Ratchet + ZK Pro
3. Periodic Anchor ($t \equiv 0 \bmod 100$)	Epoch checkpoint boundary	Sign $\text{Ed25519}_{\text{sk}}(\texttt{"anchor"} \parallel t \parallel R_t)$	179.60 μs amortized ($+1.80\,\mu$s/tu
4. Forged / Bit-Flipped Packet	τ_{ratchet} or $g^{s_t} \neq R_{\text{com}} Y_t^{e_t}$	Raise `PermissionError`; restore (R_{t-1}, t, D_{t-1})	100% reject; 0 state drift on recei
5. Replayed Packet ($t' \leq t$)	$(t', \tau_{\text{ratchet}}) \in \mathcal{W}_{\text{seen}}$ or $t' \neq t$	Reject in $\mathcal{O}(1)$ hash lookup prior to codec parse	100% replay immunity; DoS-resis
6. Goal Context Mismatch	Caller $\mathcal{G}_t \neq$ Receiver $\mathcal{G}'_t$	Divergent $H(\mathcal{G}_t) \implies R_t \neq R'_t \implies Y_t \neq \hat{Y}_t$	100% cross-goal tool hijacking bl

APPENDIX H
FORMAL SECURITY REDUCTION OF MULTI-EPOCH RATCHET INDISTINGUISHABILITY AND WATER-FILLING OPTIMALITY

Proposition 3 (Strict Concavity of Shannon Entropy and Optimality of Set-Complement Water-Filling)**.** *Let* $\mathbf{c}^{(0)} = (c_0^{(0)}, \ldots, c_{255}^{(0)}) \in \mathbb{Z}_{\geq 0}^{256}$ *be the byte-frequency histogram of the whitened prefix* W_p *of length* $L_p = \sum_{b=0}^{255} c_b^{(0)}$*, and let* $L_c = B - L_p$ *be the available chaff byte budget. Among all non-negative integer chaff allocations* $\mathbf{\Delta} = (\Delta_0, \ldots, \Delta_{255}) \in \mathbb{Z}_{\geq 0}^{256}$ *satisfying* $\sum_{b=0}^{255} \Delta_b = L_c$*, the plug-in Shannon byte entropy*

$$H(\mathbf{c}^{(0)} + \mathbf{\Delta}) = -\sum_{b=0}^{255} \frac{c_b^{(0)} + \Delta_b}{B} \log_2 \left(\frac{c_b^{(0)} + \Delta_b}{B} \right) \quad (32)$$

is Schur-concave in the total count vector $\mathbf{c} = \mathbf{c}^{(0)} + \mathbf{\Delta}$ *and is uniquely maximized (up to permutation of equal-count bins) when* $\mathbf{c}$ *is majorized by every other feasible count vector, i.e.,* $\max_b c_b - \min_b c_b \leq 1$*. Furthermore, the Stage 2B slice-wise set-complement plus* $\mathbb{Z}_{256}$ *affine-bijection construction approximates this Schur-concave optimum in* $\mathcal{O}(B)$ *C-level operations without transmitting an auxiliary histogram side channel.*

Proof. The function $\varphi(u) = -(u/B)\log_2(u/B)$ is strictly concave on $u > 0$ since $\varphi''(u) = -\frac{1}{Bu \ln 2} < 0$. By Karamata's inequality (and the theory of Schur-concave functions), a symmetric sum $\sum_{b=0}^{255} \varphi(c_b)$ of strictly concave functions subject to $\sum_{b=0}^{255} c_b = B$ is maximized when the components c_b are as nearly equal as possible. If an algorithm increments the minimum-count bins first ($c_b = 0 \to 1$, then $1 \to 2$, etc.), each discrete transfer of 1 unit from a bin with count $c_{\text{high}} \geq c_{\text{low}} + 2$ to a bin with count c_{low} strictly increases H by:

$$\Delta H = [\varphi(c_{\text{low}} + 1) - \varphi(c_{\text{low}})] - [\varphi(c_{\text{high}}) - \varphi(c_{\text{high}} - 1)] > 0. \quad (33)$$

In Stage 2B, taking the exact set complement $M^{(s)} = \Sigma_{256} \setminus \text{set}(W_p^{(s)})$ on each 128-byte slice $W_p^{(s)}$ increments every zero-count byte in that slice to 1, eliminating all zero-frequency bins in the first pass, while each subsequent 256-byte affine bijection block $\Pi_{a,r}$ adds the constant vector $\mathbf{1}_{256} = (1, 1, \ldots, 1)$ to $\mathbf{c}$, preserving the bounded variance $\sum_b (c_b - B/256)^2 \leq L_p$ while scaling $B = 256M$ and driving $H(B_t) \to 8.000$ bits/byte at rate $8 - \mathcal{O}(1/M)$. □

REFERENCES

[1] M. Luby and C. Rackoff, "How to construct pseudorandom permutations from pseudorandom functions," *SIAM Journal on Computing*, vol. 17, no. 2, pp. 373–386, 1988, doi: 10.1137/0217022.

[2] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991, doi: 10.1007/BF00196725.

[3] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology — CRYPTO '86*, Lecture Notes in Computer Science, vol. 263, pp. 186–194, Springer, 1987, doi: 10.1007/3-540-47721-7_12.

[4] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," *IETF RFC 5869*, May 2010, doi: 10.17487/RFC5869.

[5] C. Bormann and P. Hoffman, "Concise Binary Object Representation (CBOR)," *IETF STD 94, RFC 8949*, Dec. 2020, doi: 10.17487/RFC8949.

[6] A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," *IETF RFC 7748*, Jan. 2016, doi: 10.17487/RFC7748.

[7] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," *IETF RFC 8032*, Jan. 2017, doi: 10.17487/RFC8032.

[8] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," *IETF RFC 8439*, Jun. 2018, doi: 10.17487/RFC8439.

[9] M. J. Dworkin, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," National Institute of Standards and Technology (NIST), *FIPS PUB 202*, Aug. 2015, doi: 10.6028/NIST.FIPS.202.

[10] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," *IETF RFC 8446*, Aug. 2018, doi: 10.17487/RFC8446.

[11] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)," *IETF RFC 9449*, Sep. 2023, doi: 10.17487/RFC9449.

[12] A. Backman, J. Richer, and M. Sporny, "HTTP Message Signatures," *IETF RFC 9421*, Feb. 2024, doi: 10.17487/RFC9421.

[13] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proc. Network and Distributed System Security Symposium (NDSS)*, Feb. 2014, doi: 10.14722/ndss.2014.23212.

[14] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the Signal messaging protocol," *Journal of Cryptology*, vol. 33, no. 4, pp. 1914–1983, 2020, doi: 10.1007/s00145-020-09360-1.

[15] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. Network and Distributed System Security Symposium (NDSS)*, Feb. 2009.

[16] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 332–346, May 2012, doi: 10.1109/SP.2012.28.

[17] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Protocol misidentification made easy with format-transforming encryption," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 61–72, Nov. 2013, doi: 10.1145/2508859.2516657.

[18] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle, and T. Engel, "Website fingerprinting at internet scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, Feb. 2016, doi: 10.14722/ndss.2016.23477.

[19] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1928–1943, Oct. 2018, doi: 10.1145/3243734.3243768.

[20] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.

[21] J. Black and P. Rogaway, "Ciphers with arbitrary finite domains," in *Topics in Cryptology — CT-RSA 2002*, Lecture Notes in Computer Science, vol. 2271, pp. 114–130, Springer, 2002, doi: 10.1007/3-540-45760-7_9.

[22] M. Bellare, T. Ristenpart, P. Rogaway, and T. Stegers, "Format-preserving encryption," in *Selected Areas in Cryptography (SAC 2009)*, Lecture Notes in Computer Science, vol. 5867, pp. 295–312, Springer, 2009, doi: 10.1007/978-3-642-05445-7_19.

[23] J. Patarin, "Security of random Feistel schemes with 5 or more rounds," in *Advances in Cryptology — CRYPTO 2004*, Lecture Notes in Computer Science, vol. 3152, pp. 106–122, Springer, 2004, doi: 10.1007/978-3-540-28628-8_7.

[24] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989, doi: 10.1137/0218012.

[25] M. Bellare and P. Rogaway, "Random oracles are practical: A paradigm for designing efficient protocols," in *Proc. 1st ACM Conference on Computer and Communications Security (CCS)*, pp. 62–73, Nov. 1993, doi: 10.1145/168588.168596.

[26] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *Journal of Cryptology*, vol. 13, no. 3, pp. 361–396, 2000, doi: 10.1007/s001450010003.

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 79–90, Nov. 2023, doi: 10.1145/3605764.3623985.

[28] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024, arXiv:2406.13352.

[29] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics (ACL)*, pp. 10471–10506, Aug. 2024, arXiv:2403.02691.

[30] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defending LLM agents against prompt injections with CaMeL," *arXiv preprint arXiv:2503.18813*, Mar. 2025.